

Strong NP-Hardness and Approximation Algorithm for Weighted Tardiness with Release Dates and Identical Processing Times

Zhi-Long Chen

University of Maryland

Nicholas G. Hall

The Ohio State University

September 23, 2026

Abstract

We study nonpreemptive scheduling on a single machine with release dates, due dates, positive job weights, and a common processing time. The objective is to minimize total weighted tardiness. Although closely related equal-processing-time problems admit polynomial-time algorithms, the complexity of this problem has remained open in the literature since 2010. We prove that its decision version is strongly NP-complete, even when every job can meet its due date if processed immediately upon release. The reduction is from unweighted MAX-CUT and uses a quadratic number of jobs with polynomially bounded numerical data. Its main ingredient is a constructive normalization theorem that converts every sufficiently inexpensive feasible schedule into a binary choice for each graph vertex; after normalization, total weighted tardiness equals a constant minus a scaled cut value. We also give a deterministic polynomial-time phase-grid assignment algorithm for the shifted objective $\Phi = F + p \sum_j w_j$, where F is total weighted tardiness. The algorithm enumerates at most N release-date residues modulo p , solves one minimum-cost assignment problem for each residue, and returns the best phase-grid schedule. It runs in $O(N^5)$ arithmetic operations and achieves the tight ratio $3/2 - 1/(2N)$ for this algorithm. Because the added term $p \sum_j w_j$ is independent of how the jobs are scheduled, the shifted and original objectives have exactly the same optimal schedules. However, the approximation guarantee applies to the shifted objective; for the original objective, the analysis provides an additive bound. Thus, the paper both resolves the long-standing complexity question and provides a complementary worst-case guarantee for the phase-grid assignment algorithm.

Subject classifications: Analysis of algorithms: computational complexity; production/scheduling: deterministic single machine.

AMS classifications (2020): Primary 90B35; Secondary 68Q25, 90C59.

Keywords: scheduling; weighted tardiness; strong NP-hardness; approximation algorithm; assignment.

1 Introduction

We consider the problem of scheduling N jobs for processing on a single continuously available machine to minimize the total weighted tardiness where jobs have individual release dates and identical processing times. Using the three-field scheduling notation of Graham et al. (1979), where the first field describes the machine environment, the second specifies job restrictions, and the third gives the objective, this problem is denoted as $1 \mid r_j, p_j = p \mid \sum_j w_j T_j$, where each job j is associated with a release date r_j , an identical processing time $p_j = p$, a due date d_j , and a priority weight w_j . In a given schedule, the completion time of job j is denoted as C_j and its tardiness is defined as $T_j = \max\{C_j - d_j, 0\}$. Without loss of generality, it is assumed that p and w_j 's are positive integers and r_j 's and d_j 's are nonnegative integers.

Scheduling to meet due dates is a basic operations research problem. When work becomes available at different times, a decision maker must determine both which job to process next and whether to wait for a job that has not yet arrived. Total weighted tardiness measures the resulting delay by charging each job a penalty proportional to the time by which it misses its due date. The weights allow the penalty rates to differ across jobs. Equal processing times describe a natural restriction in which every job requires the same amount of machine capacity, while availability, urgency, and delay costs remain heterogeneous. This restriction often changes the computational character of a scheduling problem. It removes the possibility of encoding an arbitrary collection of item sizes directly in processing times. It also permits a polynomially bounded set of candidate start times. For several due-date objectives, these properties support dynamic programming or assignment algorithms. In particular, there are known polynomial-time algorithms for the total unweighted tardiness problem and the weighted number of tardy jobs problem with release dates and equal processing times, i.e., $1 \mid r_j, p_j = p \mid \sum_j T_j$ and $1 \mid r_j, p_j = p \mid \sum_j w_j U_j$ (Baptiste, 1999, 2000), where $U_j = \mathbf{1}_{\{C_j > d_j\}}$ is the tardy-job indicator. It is therefore natural to ask whether the total weighted tardiness problem, i.e., $1 \mid r_j, p_j = p \mid \sum_j w_j T_j$, is also tractable.

The answer has remained unresolved in the literature devoted to this problem. van den Akker et al. (2010, pp. 561 and 575) explicitly identify its computational complexity as open. Gafarov et al. (2020, p. 853) subsequently describe it as a minimal open problem in the scheduling classification and conjecture NP-hardness in their complexity discussion and concluding remarks.

We prove that the decision problem of $1 \mid r_j, p_j = p \mid \sum_j w_j T_j$ is strongly NP-complete (Theorem 1). The result holds even when every due date is at least one processing time after the corresponding release, i.e., $d_j \geq r_j + p$ for every job. Every numerical parameter in the reduction, including the objective threshold, is polynomially bounded in the number of jobs. This resolves the published complexity open question and strengthens the NP-hardness conjecture of Gafarov et al. (2020).

The proof encodes unweighted MAX-CUT. Each vertex is represented by a block of equal-length jobs with two possible normalized behaviors. A pair of jobs at each incidence determines which weight remains for processing in a final portion of the schedule. Additional equal-length jobs enforce the timing of the blocks through the objective itself. The central argument shows constructively

that any schedule meeting the reduction’s cost bound can be converted, without increasing cost, into one of these normalized schedules (Proposition 1).

The normalized objective has an exact pairwise decomposition. Its coefficients can be chosen independently, which permits a graph interaction to be represented through job weights and due dates (Proposition 2, especially (29)). This provides the needed coupling while all processing times remain identical.

The hardness result motivates the search for algorithms with provable performance. A direct multiplicative guarantee for total weighted tardiness is delicate because its optimum may be zero. We therefore analyze the shifted objective

$$\Phi(S) = F(S) + p \sum_j w_j,$$

where the added term $p \sum_j w_j$ is independent of how the jobs are scheduled. Consequently, Φ and the original objective F have the same ordering of schedules and the same minimizers. Our deterministic phase-grid assignment algorithm enumerates every distinct release-date residue modulo p . For each residue, it rounds releases upward to the associated grid, solves the resulting minimum-cost assignment problem exactly, and retains the best schedule. The algorithm uses at most N assignments, runs in $O(N^5)$ arithmetic operations, and satisfies

$$\frac{\Phi_{\text{alg}}}{\Phi^*} \leq \frac{3}{2} - \frac{\sum_j w_j^2}{2(\sum_j w_j)^2} \leq \frac{3}{2} - \frac{1}{2N}.$$

The last bound is tight for the specified phase-grid assignment algorithm. Equivalently, the analysis gives an explicit additive guarantee for F ; it does not claim a constant multiplicative approximation for the unshifted objective. The contributions are therefore the construction and normalization that encode a cut problem, together with the phase-enumeration analysis that converts related grid and assignment ideas into a worst-case guarantee for a precisely defined shifted objective.

Section 2 reviews the complexity and algorithmic results most directly surrounding the problem, including grid-based methods and shifted-objective analyses. Sections 3 and 4 give the construction and normalization argument. Section 5 derives the cost identity and proves strong NP-completeness. Section 6 presents the phase-grid assignment algorithm and its shifted-objective guarantee, and Section 7 concludes.

2 Related literature

Table 1 summarizes the complexity results of the single-machine and parallel-machine problems related to the studied problem. We briefly discuss some of the problems and results given in the table and other related literature in the following subsections.

Table 1: Complexity results surrounding equal-processing-time weighted tardiness

Problem	Result	Source
$1 \parallel \sum_j T_j$	Ordinary NP-hardness; pseudo-polynomial algorithm and FPTAS	Lawler (1977, 1982); Du and Leung (1990)
$1 \parallel \sum_j w_j T_j$	Strongly NP-hard	Lenstra et al. (1977)
$1 \mid r_j, p_j = p \mid \sum_j T_j$	Polynomial, $O(N^7)$ algorithm	Baptiste (2000)
$1 \mid r_j, p_j = p \mid \sum_j w_j U_j$	Polynomial, $O(N^7)$ algorithm	Baptiste (1999)
$1 \mid r_j, p_j = p \mid \sum_j U_j$	Polynomial, $O(N^5)$ algorithm	Chrobak et al. (2006)
$1 \mid r_j = a + k_j p, p_j = p \mid \sum_j w_j T_j$	Polynomial, reduction to assignment	Gafarov et al. (2020)
$1 \mid r_j, p_j = p \mid \sum_j w_j T_j$	Explicitly left open; NP-hardness conjectured	van den Akker et al. (2010); Gafarov et al. (2020)
Same problem, including $d_j \geq r_j + p$	Strongly NP-hard	Theorem 1
$P_m \mid r_j, p_j = p \mid \sum_j T_j$, fixed m	Polynomial, $O(N^{3m+4})$ algorithm	Baptiste (2000)
$P \mid r_j, p_j = p \mid \sum_j T_j$, input machine count	Polynomial, reduction to LP	Brucker and Kravchenko (2005)
$P_m \mid r_j, p_j = p \mid \sum_j w_j U_j$, fixed m	Polynomial, $O(N^{6m+1})$ algorithm	Baptiste et al. (2004)
$P \mid r_j, p_j = p \mid \sum_j U_j$, input machine count	NP-hard	Heeger and Molter (2025)

Note: N is the number of jobs. The symbol P in the machine field denotes an arbitrary number of identical machines; P_m denotes a given number (m) of identical machines;

2.1 Complexity results

For arbitrary processing times and no release dates, the distinction between weighted and unweighted tardiness is well established. Lawler (1977) gives a pseudo-polynomial dynamic program for $1 \parallel \sum_j T_j$, and Du and Leung (1990) prove NP-hardness in the ordinary sense. Lawler (1982) also gives a fully polynomial approximation scheme.

The unrestricted weighted counterpart $1 \parallel \sum_j w_j T_j$ is strongly NP-hard (Lenstra et al., 1977). That result does not establish hardness under a common processing time: a hardness proof for a larger class does not automatically survive the equal-duration restriction. Indeed, when the processing times are identical, i.e., $p_j = p$, if all jobs are available at time zero, the completion positions can be fixed and the jobs assigned to positions at costs $w_j \max\{kp - d_j, 0\}$. Thus, $1 \mid p_j = p \mid \sum_j w_j T_j$ reduces to an assignment problem and can be solved in polynomial time (Kuhn, 1955). This is discussed in Gafarov et al. (2020). They also point out that a more general common-grid version of the problem, where $r_j = a + k_j p$, for a common nonnegative integer a and a job dependent integer multiplier k_j , can be modeled as an assignment problem.

Baptiste (1999) establishes strongly polynomial dynamic programs for minimizing the weighted number of tardy equal-length jobs with release dates, both without and with preemption. His nonpreemptive algorithm has running time $O(N^7)$ for N jobs. Baptiste (2000) obtains polynomial algorithms for equal-length scheduling on a fixed number of identical parallel machines, including the total-tardiness objective. The single-machine result gives a polynomial algorithm for $1 \mid r_j, p_j = p \mid \sum_j T_j$. Thus replacing tardiness amounts by tardy indicators, or replacing arbitrary weights by equal weights, yields tractable neighbors of our problem.

Subsequent work develops this algorithmic boundary in several directions. Baptiste et al. (2004) give polynomial algorithms for ten equal-processing-time problems, one of which is the weighted number of tardy jobs problem on a fixed number of identical machines. Carlier (1981) gives a polynomial algorithm for deciding whether all equal-length jobs can meet their deadlines and proposes an extension to maximize throughput. Chrobak et al. (2006) identify an error in that maximization extension and provide an $O(N^5)$ algorithm for the unweighted single-machine throughput problem. Their counterexample concerns the maximization extension; the feasibility result remains valid.

Throughput and tardiness use the same release-date and due-date information but value late jobs differently. In a throughput problem, the cost associated with a job declared late is independent of how late it is. The remaining jobs can therefore be appended after the chosen on-time subset without changing the selection objective. Under total weighted tardiness, their completion times still contribute to the objective. Our reduction exploits exactly this dependence: the jobs left for the final part of the schedule carry the interaction among the earlier binary choices.

For the weighted-tardiness problem studied here, van den Akker et al. (2010) explicitly leave the general computational complexity open, and Gafarov et al. (2020) subsequently conjecture NP-hardness and reiterate the open status. Theorem 1 settles this question by proving strong NP-hardness, even under $d_j \geq r_j + p$. The result also resolves a related classification question in multiple-due-date scheduling. In the equal-processing-time model MDS-EP of Kühn et al. (2024), every job has several due dates, each carrying a penalty if missed. Their Corollary 2 gives a polynomial reduction from $1 \mid r_j, p_j = p \mid \sum_j w_j T_j$ to MDS-EP. Composing that reduction with Theorem 1 proves that MDS-EP is NP-hard; this observation does not assert preservation of strong NP-hardness or approximation guarantees.

Parallel-machine classifications depend on the number of machines and on preemption assumptions (Kravchenko and Werner, 2011). Equal-length total tardiness is polynomial for a fixed number of identical machines (Baptiste, 2000), and equal-length weighted throughput is polynomial for a fixed number of nonpreemptive machines (Baptiste et al., 2004). When the number of machines is part of the input, hardness is known for the preemptive weighted-count problem (Brucker and Kravchenko, 2006) and for the nonpreemptive unweighted-count problem with release dates (Heeger and Molter, 2025); the latter resolves the question discussed by Sgall (2012). These parallel-machine results concern tardy-job counts or unweighted tardiness under different machine assumptions, rather than the single-machine weighted-tardiness objective considered here.

2.2 Exact, heuristic, and grid-based algorithms

Exact and heuristic procedures for the more general problem $1 \mid r_j \mid \sum_j w_j T_j$ also apply when processing times are equal. Aktürk and Özdemir (2000) develop a branch-and-bound method based on time-dependent dominance rules and lower bounds, and Aktürk and Özdemir (2001) use related dominance information in constructive and local-search heuristics. These methods do not exploit the common processing time through residue grids or assignment. For $1 \mid r_j, p_j = p \mid \sum_j w_j T_j$, van

den Akker et al. (2010) develop a time-indexed formulation over the union of the release-generated grids, together with LP-based structural arguments, branching, and column generation. They report tractable special cases with common due dates, common weights, common releases, or equally ordered release and due dates. Their formulation retains capacity interactions among grids with different residues, whereas our algorithm separates the residue phases and solves one assignment problem for each phase. Their computational procedures address exact solution of instances, but favorable behavior of the LP relaxation does not provide a polynomial worst-case bound for the general problem.

For the equal-processing-time problem itself, Gafarov et al. (2020) give two exact algorithms with running time $O(N^3 2^N)$. One enumerates subsets of jobs fixed to start at their release dates and solves an assignment problem for the remaining jobs. This has the same high-level pattern as our method—enumerate a structural choice, solve an assignment problem, and select the best solution—but its enumeration is exponential and exact, whereas our algorithm examines at most N residue phases and is polynomial and approximate. They also study dominance properties, special cases, and exact search, and question the completeness of an LP-conversion argument in the earlier work. Our hardness proof and approximation analysis do not rely on that conversion argument.

Time grids and assignment reductions also occur in neighboring due-date problems. Dessouky et al. (1990) reduce identical-job scheduling without individual release restrictions to linear assignment, obtaining an $O(N^3)$ algorithm for general nondecreasing costs and $O(N \log N)$ algorithms for total tardiness and the weighted number of tardy jobs. Zhang et al. (2025) use generated completion times for bicriterion equal-length scheduling involving total tardiness and the number of tardy jobs on uniform parallel machines, with an $O(N^2)$ Pareto algorithm and $O(N \log N)$ algorithms for two hierarchical variants. The polynomial dynamic programs summarized in the preceding subsection likewise restrict attention to times of the form $r_i + kp$, but optimize jointly over the union of all release-generated grids. In contrast, our algorithm solves an assignment problem independently for each residue class and then compares the phase optima. Gafarov et al. (2020) show that selecting one arbitrary grid can have unbounded relative error for the original objective; Section 6 obtains a bounded additive loss by testing every release phase and evaluates that loss under a shifted objective.

2.3 Shifted-objective performance guarantees

There is direct precedent for using a translated objective when tardiness can vanish. For $P_m \mid d_j = d \mid \sum_j T_j$, Kovalyov and Werner (2002) construct approximation schemes satisfying

$$\frac{T^A - T^*}{T^* + d} \leq \varepsilon, \quad \text{equivalently} \quad \frac{T^A + d}{T^* + d} \leq 1 + \varepsilon.$$

Thus the common due date is added explicitly to the performance measure. For fixed m , their two algorithm families run in $O(N^{2m}/\varepsilon^{m-1})$ and $O(N^{m+1}/\varepsilon^m)$ time; for $m = 2$, the first improves to $O(N^3/\varepsilon)$.

Kolliopoulos and Steiner (2006) study the broader family $\alpha \mid \beta \mid \sum_j w_j(T_j + d_j)$ on single, identical-parallel, uniformly related, and unrelated machines, with variants involving release dates, precedence constraints, and preemption. Their models include, for example, $P \mid r_j \mid \sum_j w_j(T_j + d_j)$, $1 \mid r_j, \text{prec} \mid \sum_j w_j(T_j + d_j)$, and $R \mid r_j \mid \sum_j w_j(T_j + d_j)$. They transfer approximation results from weighted completion time and give an $O(N^3/\varepsilon)$ FPTAS for $1 \mid d_j = D \mid \sum_j w_j(T_j + d_j)$. Since $\sum_j w_j(T_j + d_j) = \sum_j w_j T_j + \sum_j w_j d_j$, the modification preserves the optimal schedules while changing the approximation ratio.

A related transformation is standard for maximum lateness. If $D \geq \max_j d_j$ and $q_j = D - d_j$, then $\max_j \{C_j + q_j\} = D + L_{\max}$. In the release-date/delivery-time formulation, Potts (1980) give a $3/2$ -approximation for $1 \mid r_j \mid L_{\max}$, and Mastrolilli (2003) develop PTASs for $P \mid r_j \mid L_{\max}$ and its single-machine special case. These two results apply to the delivery completion objective $Q_{\max}$, which is equivalent to $D + L_{\max}$ under the displayed transformation. These studies support the use of a schedule-independent shift as a performance measure, but the guarantee must be identified as applying to the translated objective, not as an ordinary multiplicative approximation for unshifted tardiness.

2.4 Position of our result

Our complexity result establishes strong NP-hardness while retaining one machine, equal job lengths, arbitrary release dates, and the usual weighted-tardiness objective. It strengthens the published NP-hardness conjecture and separates this model from both the unweighted total-tardiness problem and the weighted-throughput problem. Complementing that negative result, Section 6 combines residue-class enumeration and assignment with a shifted-objective analysis. We found no previous algorithm with this combination for $1 \mid r_j, p_j = p \mid \sum_j w_j T_j$. The complexity conclusion does not contradict the usefulness of existing exact methods on structured or moderate-size instances, and the shifted guarantee does not imply a multiplicative approximation for the original objective.

Several ingredients remain standard and are credited where used: the scheduling notation, candidate-time dominance, bipartite assignment, the weighted completion-time interchange rule of Smith (1956), and the NP-completeness of unweighted MAX-CUT established by Garey et al. (1976). The new contributions are the equal-length scheduling construction, its normalization for arbitrary bounded-cost witnesses, its exact encoding of independently specified positive edge coefficients, and the phase-averaging analysis of the phase-grid assignment algorithm.

3 Reduction from unweighted MAX-CUT

In a feasible schedule, each job j , for $j = 1, \dots, N$, is executed in $[s_j, s_j + p)$, where $s_j \geq r_j$. The intervals are pairwise disjoint. Preemption is forbidden and idle time is permitted. The objective is

$$\sum_{j=1}^N w_j \max\{0, C_j - d_j\}, \quad \text{where } C_j = s_j + p.$$

The decision problem asks whether this cost is at most a given nonnegative integer K . All numerical input is binary encoded. Start times may be real. The common processing time is part of the instance, and can vary between instances.

The main strong NP-completeness result is stated and proved in Section 5.

We first record the left-shifting argument underlying candidate-time discretization in equal-length scheduling (Baptiste et al., 2004; van den Akker et al., 2010). It also specifies why allowing real start times does not invalidate the later integral arguments. For any feasible schedule, let π be its job permutation. Define its earliest schedule by

$$\hat{C}_0 = 0, \quad \hat{s}_{\pi(k)} = \max\{r_{\pi(k)}, \hat{C}_{k-1}\}, \quad \hat{C}_k = \hat{s}_{\pi(k)} + p. \quad (1)$$

Here k is the position in π , and $\hat{C}_k$ is the completion time of job $\pi(k)$ in the earliest schedule. For $k = 1$, feasibility gives $s_{\pi(1)} \geq r_{\pi(1)}$, so (1) gives $\hat{s}_{\pi(1)} \leq s_{\pi(1)}$. For $k > 1$, assume $\hat{C}_{k-1} \leq C_{\pi(k-1)}$. Feasibility gives $s_{\pi(k)} \geq r_{\pi(k)}$ and $s_{\pi(k)} \geq C_{\pi(k-1)}$; hence (1) gives $\hat{s}_{\pi(k)} \leq s_{\pi(k)}$. Adding p proves $\hat{C}_k \leq C_{\pi(k)}$ for every k . The new schedule is feasible, has integer starts, and has no greater cost, because tardiness is nondecreasing in completion time. This argument applies to every feasible schedule, including one with noninteger starts, without any optimality assumption. In particular, the minimum over the finitely many earliest permutations attains the optimum. All uses of a one-unit lower bound on positive tardiness below explicitly require an integral schedule, obtained by (1) when necessary.

The source problem for the reduction is unweighted MAX-CUT (Garey et al., 1976). The reduction encodes each vertex by a block of equal-length jobs. For every vertex–edge incidence, the construction creates two alternative jobs. In the schedules that survive the normalization argument, exactly one of these two jobs is processed in the corresponding vertex block and the other is postponed to a final tail. This binary choice represents the side of the cut to which the vertex is assigned. The remaining jobs and parameters are chosen so that the block structure is enforced, while the weighted tardiness contributed by the tail decreases exactly when the endpoints of an edge are assigned to different sides of the cut.

Section 3.1 gives the full construction of the scheduling instance, including the auxiliary complete graph, the incidence indexing, the job data, the block boundaries, and the decision threshold. Section 3.2 records the parameter inequalities that are used later in the schedule argument. These inequalities ensure that the intended block structure is forced for every schedule whose cost is below the reference bound. Section 4 then shows that such schedules can be transformed into a canonical form, and Section 5 evaluates the cost of canonical schedules to obtain the correspondence with MAX-CUT.

3.1 Source instance and job data

Take an unweighted simple graph $\mathcal{G} = (\mathcal{V}, \mathcal{F})$, where $\mathcal{V} = \{1, \dots, n\}$ is the vertex set $\mathcal{F}$ the edge set, and an integer κ . MAX-CUT asks whether some bipartition cuts at least κ edges. To avoid

trivial instances of the problem, we assume that there are no isolated vertices in $\mathcal{G}$, and $1 \leq \kappa \leq |\mathcal{F}|$. Hence, $2 \leq n \leq 2|\mathcal{F}|$.

Auxiliary complete graph. The construction uses the auxiliary complete graph $K_n = (\mathcal{V}, \mathcal{E})$, with

$$\mathcal{E} = \{\{a, b\} : 1 \leq a < b \leq n\}.$$

Edges in $\mathcal{F}$ are called *original edges*; edges in $\mathcal{E} \setminus \mathcal{F}$ are called *added edges*. No vertices are added. The MAX-CUT objective counts only original edges, whereas every edge of the auxiliary complete graph participates in the scheduling construction. Define

$$\Gamma = \left\lfloor \frac{n^2}{4} \right\rfloor + 1, \quad \Delta_e = 1 + \Gamma \mathbf{1}_{\{e \in \mathcal{F}\}} \quad (e \in \mathcal{E}). \quad (2)$$

Thus $\Delta_e = \Gamma + 1$ for an original edge and $\Delta_e = 1$ for an added edge. Put $m = n - 1$, $M = nm$, and $E = |\mathcal{E}| = M/2 = \binom{n}{2}$. Each vertex will be associated with a *vertex block*, defined below as a time interval.

Edge indices and weight intervals. Write each unordered edge as $e = \{a, b\}$ with $a < b$, order $\mathcal{E}$ lexicographically by $t_e \in \{1, \dots, E\}$, and define

$$\ell_e = t_e + \sum_{\substack{f \in \mathcal{E} \\ t_f < t_e}} \Delta_f, \quad h_e = \ell_e + \Delta_e \quad (e \in \mathcal{E}). \quad (3)$$

The sum is over all edges preceding e in this order, including original and added edges. The intervals $[\ell_e, h_e]$ are disjoint, with a one-unit gap between successive intervals. Their maximum endpoint is

$$V = E + \sum_{e \in \mathcal{E}} \Delta_e. \quad (4)$$

The scalar V is distinct from the vertex set $\mathcal{V}$.

Vertex-edge incidences and their ranks. For each vertex $v \in \mathcal{V}$, let

$$\mathcal{E}_v = \{e \in \mathcal{E} : v \in e\} = \{\{a, v\} : 1 \leq a < v\} \cup \{\{v, b\} : v < b \leq n\}.$$

The set $\mathcal{E}_v$ contains all $m = n - 1$ edges incident to v , with both smaller and larger other endpoints. A *vertex-edge incidence* (v, e) records a vertex v and an edge $e \in \mathcal{E}_v$ containing it. An edge $e = \{a, b\} = \{b, a\}$ has two incidences, (a, e) and (b, e) . There are $M = 2E$ incidences in total. Each incidence will receive its own job pair, hence one edge gives two such job pairs, one at each endpoint.

At a fixed vertex v , order its incident edges by increasing other endpoint. For $e = \{a, b\}$ with $a < b$, the local incidence rank and global incidence rank are defined, respectively,

$$k_{v,e} = \begin{cases} b-1, & v = a, \\ a, & v = b, \end{cases} \quad \text{and} \quad g_{v,e} = (v-1)m + k_{v,e}.$$

Local ranks run from 1 to m . The absence of a loop $\{v, v\}$ explains the two cases in the formula: for $v = a$, the other endpoint b has rank $b-1$, whereas for $v = b$, the other endpoint a has rank a . Equivalently, the local list is ordered by increasing other endpoint. Global ranks run from 1 to M , with these local lists concatenated in increasing vertex order. Thus the vertex is ordered first and, within that vertex, the incident edges are ordered by increasing other endpoint (equivalently, by the lexicographic order of their canonical labels restricted to $\mathcal{E}_v$). Write $e(g)$ for the unique edge associated with incidence rank g . For each edge e , its two incidences share the same quantities $t_e, \Delta_e, \ell_e, h_e$.

Table 2 displays the rank pattern for general n .

Table 2: Local and global incidence ranks for general n

Vertex v	Incident edges in local order	Local ranks $k_{v,e}$	Global ranks $g_{v,e}$
1	$\{1, 2\}, \{1, 3\}, \dots, \{1, n\}$	$1, 2, \dots, n-1$	$1, 2, \dots, n-1$
2	$\{1, 2\}, \{2, 3\}, \dots, \{2, n\}$	$1, 2, \dots, n-1$	$n, n+1, \dots, 2n-2$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	$\{1, i\}, \dots, \{i-1, i\}, \{i, i+1\}, \dots, \{i, n\}$	$1, 2, \dots, n-1$	$(i-1)(n-1)+1, \dots, i(n-1)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	$\{1, n\}, \{2, n\}, \dots, \{n-1, n\}$	$1, 2, \dots, n-1$	$(n-1)^2+1, \dots, M$

For every vertex v , ordering the other endpoints increasingly while omitting v gives the same set of local ranks $\{1, \dots, n-1\}$. Equivalently, for a neighbor $u \neq v$,

$$k_{v,\{u,v\}} = \begin{cases} u, & u < v, \\ u-1, & u > v. \end{cases}$$

In the general row of Table 2, ranks $1, \dots, i-1$ correspond to the lower-numbered neighbors of i , and ranks $i, \dots, n-1$ correspond to its higher-numbered neighbors. Global ranks depend on the vertex: the incidences of vertex v occupy the consecutive block

$$\{(v-1)(n-1)+1, \dots, v(n-1)\}.$$

These n blocks partition $\{1, \dots, M\}$.

Charge coefficients and scaling. Define

$$P_0 = 2M, \quad L_0 = 2, \quad q_e = M - 2t_e \quad (e \in \mathcal{E}),$$

and, for every vertex $v \in \mathcal{V}$, let

$$f_v = \sum_{e \in \mathcal{E}_v} \Delta_e \left[P_0 \left(q_e + \frac{3}{2} \right) + L_0(g_{v,e} - 1) \right]. \quad (5)$$

The coefficients are chosen so that, in the cost calculation for the normalized schedules, all terms depending on only one endpoint choice cancel out. What remains is a constant baseline together with a pairwise saving that occurs exactly when the two endpoint choices differ. Proposition 2 below verifies this cancellation explicitly after scaling.

Every q_e is a nonnegative even integer. This, together with the fact that P_0 is even and $g_{v,e}$ is a positive integer, implies that all f_v are positive integers. Write

$$f_{\min} = \min_{1 \leq v \leq n} f_v$$

and set

$$U = 1 + V(2M + 1)P_0 + \sum_{v=1}^n f_v, \quad (6)$$

$$G = \sum_{\substack{e=\{a,b\} \in \mathcal{E} \\ a < b}} (U + h_e) [P_0(2q_e + 3) + L_0(g_{a,e} + g_{b,e} - 2)], \quad (7)$$

$$Q = \left\lfloor \frac{G}{2P_0 f_{\min}} \right\rfloor + 1. \quad (8)$$

In (7), the sum defining G includes every original and added edge of $\mathcal{E}$ exactly once. Finally, let

$$P = QP_0, \quad L = QL_0, \quad F_v = Qf_v \quad (v \in \mathcal{V}), \quad F_{\text{sum}} = \sum_{v=1}^n F_v, \quad \bar{K} = QG. \quad (9)$$

The sum defining F_{sum} includes one charge weight for every vertex. The parameters are defined in the displayed order; no optimum or unknown subset is used in calculating them.

Block boundaries and jobs. The n vertex blocks are time intervals and therefore have $n + 1$ boundary times. Define

$$b_s = (s - 1)((m + 2)P + 1) \quad (1 \leq s \leq n + 1), \quad H = b_{n+1}. \quad (10)$$

Vertex $v \in \mathcal{V}$ has block $[b_v, b_{v+1})$. Thus $b_1 = 0$, and $b_{n+1} = H$ is the end of the last vertex block, not an additional vertex. It is also the due date of the last guard and the starting time of the normalized tail.

For each incidence $g = g_{v,e} = (v-1)m + k_{v,e}$, introduce two jobs A_g, B_g . Add a charge job Z_v and a guard job R_v for each vertex $v \in \mathcal{V}$. Every job has processing time P . Their release dates, due dates, and weights are specified in Table 3. In particular, the two jobs A_g, B_g associated with incidence g have common due date $d_g = H - (g-1)L$. There are $2M + 2n = 2n^2$ jobs.

Table 3: Job data in the reduction; every job has processing time P

Job	Release	Due date	Weight	Number
A_g	$b_v + (k-1)P$	$H - (g-1)L$	$U + \ell_e$	M
B_g	$b_v + (k-1)P + 1$	$H - (g-1)L$	$U + h_e$	M
Z_v	$b_v + mP$	$b_v + (m+1)P$	F_v	n
R_v	$b_v + (m+1)P + 1$	b_{v+1}	$\bar{K} + 1$	n

In the A_g and B_g rows, $v \in \mathcal{V}$, $e \in \mathcal{E}_v$, $k = k_{v,e} \in \{1, \dots, m\}$, and $g = g_{v,e} = (v-1)m + k \in \{1, \dots, M\}$. Each incidence supplies one A_g -job and one B_g -job. In the charge and guard rows, $v = 1, \dots, n$.

The target threshold is

$$K_* = \bar{K} - \frac{P}{2}\Gamma\kappa. \quad (11)$$

All job data and the bound $\bar{K}$ depend only on the graph. The MAX-CUT target κ enters the nontrivial scheduling construction solely through the decision threshold K_* .

3.2 Bounds needed for the schedule argument

Lemma 1 (Parameter bounds). *The parameters defined in Section 3.1 satisfy*

$$P = ML > 1, \quad (12)$$

$$\left(\min_{1 \leq i \leq n} F_i \right) (2P + 1) > \bar{K}, \quad (13)$$

$$UL > V(2M + 1)P + F_{\text{sum}}, \quad (14)$$

$$PU > F_{\text{sum}}. \quad (15)$$

Proof. Recall that $P_0 = 2M$, $L_0 = 2$, $Q \geq 1$, and $M = n(n-1) \geq 2$. Therefore, from (9),

$$P = QP_0 = 2QM = MQL_0 = ML > 1,$$

which proves (12).

For (13), Equation (8) and $f_{\min} > 0$ give

$$Q = \left\lfloor \frac{G}{2P_0 f_{\min}} \right\rfloor + 1 > \frac{G}{2P_0 f_{\min}}. \quad (16)$$

Hence, using $\min_{1 \leq i \leq n} F_i = Qf_{\min}$, $P = QP_0$, and $\bar{K} = QG$ from the scaling definitions (9),

$$\begin{aligned} \left(\min_{1 \leq i \leq n} F_i \right) (2P + 1) &= Qf_{\min}(2QP_0 + 1) \\ &> 2Q^2P_0f_{\min} \\ &> QG, \quad \text{from (16)} \\ &= \bar{K}. \end{aligned}$$

Let $\delta = 2P_0f_{\min}Q - G$. By the definition of Q given in (8), we have $1 \leq \delta \leq 2P_0f_{\min}$. The exact margin is

$$\begin{aligned} \left(\min_{1 \leq i \leq n} F_i \right) (2P + 1) - \bar{K} &= Q[2P_0f_{\min}Q + f_{\min} - G] \\ &= Q(\delta + f_{\min}) > 0. \end{aligned} \tag{17}$$

For (14), Equation (6) gives

$$V(2M + 1)P_0 + \sum_{v=1}^n f_v = U - 1.$$

Using $L = 2Q$, $P = QP_0$ and $F_{\text{sum}} = Q \sum_{i=1}^n f_i$ from (9), we obtain

$$V(2M + 1)P + F_{\text{sum}} = Q(U - 1) < 2QU = UL.$$

The exact margin, which is used below, is

$$\begin{aligned} UL - V(2M + 1)P - F_{\text{sum}} &= Q \left[2U - V(2M + 1)P_0 - \sum_{i=1}^n f_i \right] \\ &= Q \left[2 + V(2M + 1)P_0 + \sum_{i=1}^n f_i \right] \\ &= Q(U + 1) > 0. \end{aligned} \tag{18}$$

Finally, Equation (6) gives $U > \sum_{i=1}^n f_i$. Since $P_0 \geq 1$ and $Q > 0$,

$$PU = QP_0U \geq QU > Q \sum_{i=1}^n f_i = F_{\text{sum}},$$

from (6) and (9), and proving (15). Equivalently, the exact margin is $PU - F_{\text{sum}} = Q(P_0U - \sum_{i=1}^n f_i) > 0$. $\square$

The latest release among the A/B -jobs is that of B_M , whose local rank is m at vertex n . By the B_g row of Table 3 and the block definition (10), its release date is

$$r_{\max}^{AB} = b_n + (m - 1)P + 1 = H - 3P < H. \tag{19}$$

In particular, every A/B -job is available at time H , and the latest such release plus P is $H - 2P$. By Table 3, the earliest due date occurs at $g = M$; using $(M - 1)L = P - L$ from (12), it is

$$H - (M - 1)L = H - P + L > H - P. \quad (20)$$

Thus $d_j \geq r_j + P$ for every A/B -job, and equality holds for the charge and guard jobs. Indeed, (19) and (20) give $r_j + P \leq H - 2P < H - P < d_j$ for every A/B -job, while the equalities for charge and guard jobs follow from Table 3 and (10).

We next give an explicit reference schedule with total weighted tardiness $\overline{K}$. This schedule is not claimed to be optimal; its role is to show that the constructed instance has a feasible schedule of cost at most $\overline{K}$, and hence the class of schedules considered in the normalization argument is nonempty. It also gives the baseline cost from which the later cut-dependent savings are measured.

Let σ be a permutation of $\{1, \dots, M\}$ such that

$$w_{B_{\sigma(1)}} \geq \dots \geq w_{B_{\sigma(M)}},$$

with ties broken arbitrarily. Its job sequence is

$$(A_1, \dots, A_m, Z_1, R_1, A_{m+1}, \dots, A_{2m}, Z_2, R_2, \dots, \\ A_{(n-1)m+1}, \dots, A_M, Z_n, R_n, B_{\sigma(1)}, \dots, B_{\sigma(M)}).$$

The associated start times are

$$\begin{aligned} s_{A_{(v-1)m+k}} &= b_v + (k - 1)P & (1 \leq v \leq n, 1 \leq k \leq m), \\ s_{Z_v} &= b_v + mP & (1 \leq v \leq n), \\ s_{R_v} &= b_v + (m + 1)P + 1 & (1 \leq v \leq n), \\ s_{B_{\sigma(t)}} &= H + (t - 1)P & (1 \leq t \leq M). \end{aligned}$$

Thus there is one unit of idle time between the completion of each Z_v and the start of R_v . Each guard finishes at b_{v+1} , and the last guard finishes at H . All A -jobs, charge jobs, and guard jobs are on time. The final suffix after time H consists of the B -jobs; informally, this is the tail of the reference schedule. For each edge $e = \{a, b\}$, the two tail jobs are $B_{g_{a,e}}$ and $B_{g_{b,e}}$, both of weight $U + h_e$ (Table 3). The weight intervals increase with t_e by (3), so exactly $2(E - t_e) = M - 2t_e = q_e$ tail jobs have greater weight. Hence these two jobs occupy tail positions $q_e + 1$ and $q_e + 2$, with completion times $H + (q_e + 1)P$ and $H + (q_e + 2)P$, in either order. Their due dates are $H - (g_{a,e} - 1)L$

and $H - (g_{b,e} - 1)L$. Both jobs complete after H and have due dates at most H , so $T_j = C_j - d_j$ for each of them. Their total tardiness is therefore

$$\begin{aligned} & w_{B_{g_{a,e}}} T_{B_{g_{a,e}}} + w_{B_{g_{b,e}}} T_{B_{g_{b,e}}} \\ &= (U + h_e) \left[2H + (2q_e + 3)P - (2H - L(g_{a,e} + g_{b,e} - 2)) \right] \\ &= (U + h_e) [P(2q_e + 3) + L(g_{a,e} + g_{b,e} - 2)]. \end{aligned}$$

The first equality subtracts the sum of their due dates from the sum of their completion times; their order is immaterial because their weights are equal.

Using (7) and (9) gives

$$\begin{aligned} W_{\text{ref}} &= \sum_{\substack{e=\{a,b\} \in \mathcal{E} \\ a < b}} (U + h_e) [P(2q_e + 3) + L(g_{a,e} + g_{b,e} - 2)] \\ &= \sum_{\substack{e=\{a,b\} \in \mathcal{E} \\ a < b}} (U + h_e) [QP_0(2q_e + 3) + QL_0(g_{a,e} + g_{b,e} - 2)] \\ &= \sum_{\substack{e=\{a,b\} \in \mathcal{E} \\ a < b}} Q(U + h_e) [P_0(2q_e + 3) + L_0(g_{a,e} + g_{b,e} - 2)] \\ &= Q \sum_{\substack{e=\{a,b\} \in \mathcal{E} \\ a < b}} (U + h_e) [P_0(2q_e + 3) + L_0(g_{a,e} + g_{b,e} - 2)] \\ &= QG \\ &= \overline{K}. \end{aligned}$$

This complete schedule processes every job exactly once and proves that the optimal objective value of the constructed instance $\text{OPT} \leq \overline{K}$. The smaller value $K_* = \overline{K} - (P/2)\Gamma\kappa$ is the decision threshold, not the cost of this reference schedule and not necessarily the optimum. Cut-dependent schedules may meet or improve on K_* by choosing different modes at different vertices.

4 Schedule normalization

This section proves that every feasible schedule of cost at most $\overline{K}$ can be transformed, in polynomial time and without increasing cost, into a highly structured canonical schedule. Consequently, for the decision threshold $K_* \leq \overline{K}$, it suffices to analyze canonical schedules. We first determine the positions of the guard and charge jobs and characterize the feasible sets of early A/B -jobs. We then use exchange arguments to obtain exactly one early job from each pair (A_g, B_g) and to enforce a consistent A - or B -mode at every vertex. These steps culminate in a normalization theorem showing that every bounded-cost schedule, including one with noninteger start times, can be transformed in polynomial time and without increasing cost into a canonical schedule, as established in Proposition 1. This structure will allow Section 5 to encode a schedule by one binary

choice per vertex and calculate its cost edge by edge. It applies to every schedule meeting the decision threshold because $K_* \leq \overline{K}$ by (11).

Figure 1 summarizes the normalization steps and their proof references, followed by the cost identity and threshold equivalence.

Lemma 2 (Control placement). *In any integral feasible schedule of cost at most $\overline{K}$, for every $i \in \mathcal{V}$, guard job R_i starts at its release date, while charge job Z_i starts at either $b_i + mP$ or $b_i + mP + 1$ and has weighted-tardiness cost zero or F_i , respectively.*

Proof. By Table 3 and (10), $d_{R_i} = r_{R_i} + P$ and $w_{R_i} = \overline{K} + 1$. Thus, if a guard job were late in the integral schedule, it would cost at least $\overline{K} + 1$. Every other cost is nonnegative, so this would exceed the schedule's cost bound. Thus it occupies its prescribed interval.

Let $s = b_i + (m + 1)P + 1$ be its start. Charge Z_i has release $s - P - 1$ and due date $s - 1$. If the charge does not finish by s , nonpreemption and the fixed guard imply it cannot start before $s + P$ and cannot finish before $s + 2P$. Its tardiness would be at least $2P + 1$, contradicting (13). Integral feasibility before s leaves exactly the two stated charge starts. $\square$

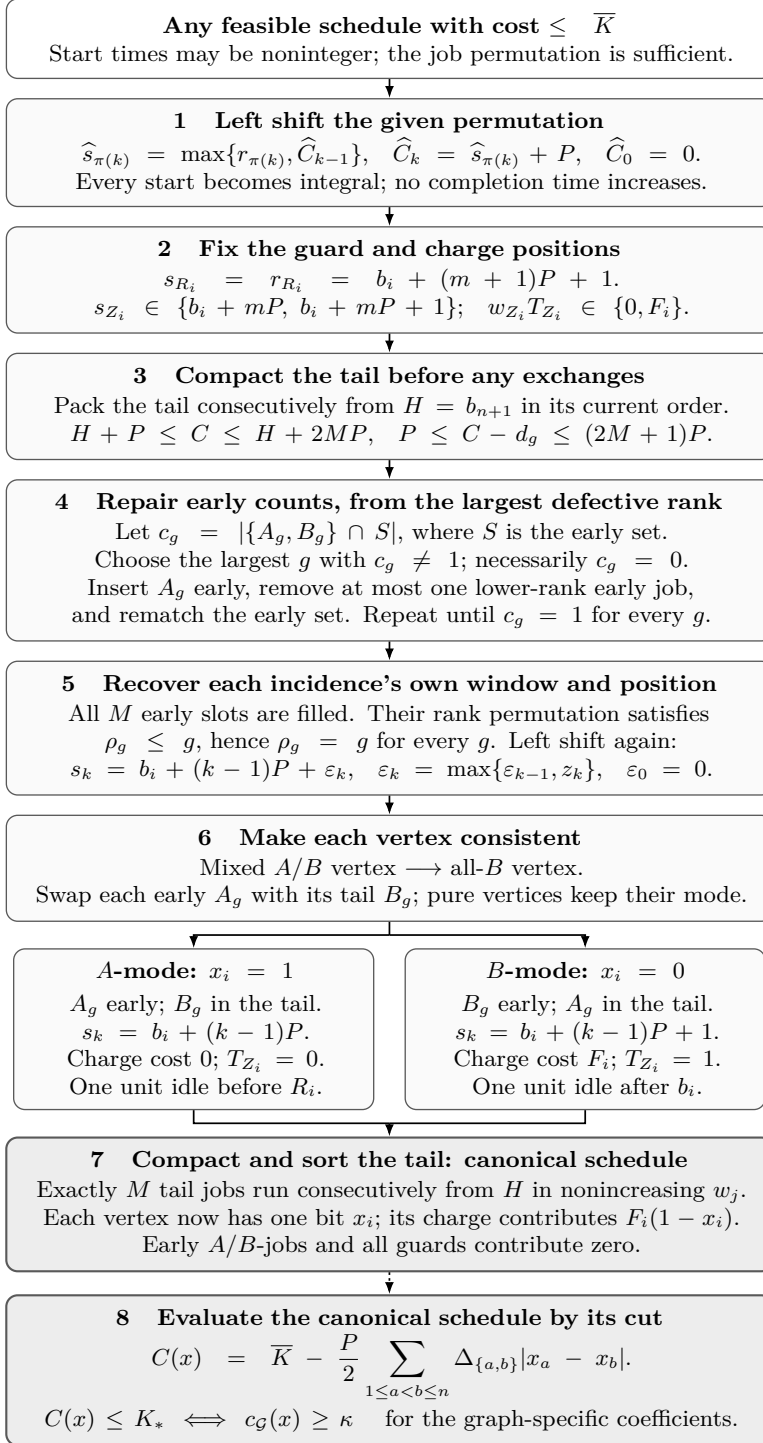
For an integral schedule of cost at most $\overline{K}$, consider the interval in vertex block i before the charge job starts. Lemma 2 places the charge start at either $b_i + mP$ or $b_i + mP + 1$, so this interval has length at most $mP + 1$ and hence fits at most m A/B -jobs. The gap after a charge and before its guard is at most one unit, less than P . The last guard occupies $[H - P, H)$, so no A/B -job can straddle H . Consequently, every A/B -job executed before H is in an early window and completes no later than the last charge start $b_n + mP + 1 = H - 2P$; by (20) it is on time. Every other A/B -job starts at or after H and is tardy, since Table 3 gives $d_g \leq H$. Call every A - or B -job processed at or after H a *tail job*. Thus the tail jobs are precisely the A/B -jobs not processed in an early window; charge and guard jobs are not included. By (19), all tail jobs have release date at most $H - 3P < H$. We may therefore compact them consecutively from H , retaining their order and leaving the early part and charge and guard jobs unchanged; this cannot increase cost. Since there are $2M$ A/B -jobs in total, there are at most $2M$ tail jobs, so their completion times then satisfy

$$H + P \leq C \leq H + 2MP, \quad P \leq C - d_g \leq (2M + 1)P. \quad (21)$$

The completion-time bounds use the preceding compaction and the fact that there are at most $2M$ tail jobs. For a tail A/B -job with incidence rank g , Table 3 gives $C - d_g = (C - H) + (g - 1)L$. By (12), $0 \leq (g - 1)L \leq (M - 1)L = P - L < P$, which gives the stated bounds on the job's tardiness $C - d_g$. These bounds do not hold for an arbitrary uncompressed tail; compaction supplies the finite horizon before any exchange is used. The same bounds remain valid if some of these tail slots are later left empty. We use this observation during the exchanges below.

Overview of the proof

Steps 1–7 preserve feasibility and never increase total weighted tardiness.



Legend: proof references

Input schedule and normalization bound.
 $K_* \leq \bar{K}$: (11).

1 Proposition 1, proof
Applied through Lemma 4; the earliest-permutation recurrence is (1).

2 Lemma 2: Control placement
Integrality and the cost bound force these positions. Each early window holds at most m jobs.

3 Compaction before Lemma 3
That paragraph proves (21); (19) puts every tail release before H .

4 Lemma 4: One early job per incidence
Lemma 3 supplies matching via the suffix condition (22):
 $\sum_{h=g}^M c_h \leq M - g + 1$.
At most M repairs; cost decreases.

5 Lemma 5, proof: rank forcing
Capacity and releases force $\rho_g = g$.
Here s_k is the local early start, $z_k = 0$ for a selected A and $z_k = 1$ for a selected B ; $\varepsilon_k \in \{0, 1\}$.

6 Lemma 5: Vertex consistency
A mixed vertex already pays F_i . The charge cost stays fixed, while the tail saving in (24) is strictly positive.

Two resulting modes, independently by vertex.
In these boxes, $g = (i-1)m + k$ and $1 \leq k \leq m$. Both modes have
 $s_{R_i} = b_i + (m+1)P + 1$
and $C_{R_i} = b_{i+1}$: the guard resets the next block.

7 Proposition 1: Normalization
For the fixed tardy tail set, minimize $\sum w_j C_j$ by sorting weights. The entire transformation is polynomial and does not assume optimality.

8 Proposition 2: Cost identity
Equation (29); then **Theorem 1** gives the threshold equivalence. Polynomial numerical bounds and membership in NP complete strong NP-completeness.

Solid arrows show normalization; branches show possible vertex modes; the dashed arrow applies the cost identity.

Figure 1: Normalization of a bounded-cost schedule. Steps 1–7 preserve feasibility and do not increase total weighted tardiness; Step 8 applies the cost identity and the MAX-CUT threshold equivalence. The legend identifies the supporting results.

Lemma 3 (Nested slot matching). *Under the charge and guard placements of Lemma 2, let S be a selected set of early A/B -jobs and let $c_g \in \{0, 1, 2\}$ count its members from the job pair (A_g, B_g) , for $1 \leq g \leq M$. If S is schedulable before the charge jobs, then necessarily*

$$\sum_{h=g}^M c_h \leq M - g + 1 \quad (1 \leq g \leq M). \quad (22)$$

Conversely, every selected set S whose counts satisfy (22) can be scheduled in the following early slots: slot $g = (i - 1)m + k$ starts at

$$b_i + (k - 1)P + 1, \quad 1 \leq i \leq n, \quad 1 \leq k \leq m, \quad (23)$$

with all charges one unit late and all guards at their release dates.

Proof. From the release date $b_i + (k - 1)P$ associated with incidence rank $g = (i - 1)m + k$ onward, its vertex window has length at most $(m - k + 1)P + 1$ by Lemma 2. It therefore fits at most

$$\left\lfloor \frac{(m - k + 1)P + 1}{P} \right\rfloor = m - k + 1$$

jobs, since $P > 1$ by (12). Each later window fits at most m . Neither member of a job pair ranked at least g can run before this release date, by the release dates in Table 3 and the ordering of the global incidence ranks. The total capacity is at most $(m - k + 1) + (n - i)m = M - g + 1$. This proves (22); any older jobs occupying these windows can only reduce the available capacity.

Conversely, the empty-set case uses no early A/B -job slots. Otherwise let the chosen incidence ranks, with one entry for each selected job, be $u_1 \leq \dots \leq u_s$. The suffix inequality at u_t gives $s - t + 1 \leq M - u_t + 1$, so $u_t \leq M - s + t$. Assign job t to slot number $M - s + t$ in (23), for $1 \leq t \leq s$. Its incidence rank is at most that slot number; thus its release date is satisfied, even if it is a B -job. Slots do not overlap. The last slot in each vertex block completes at its proposed charge-job start, and the charge job completes at its guard's start. The latest early-slot completion in (23) is $H - 2P$, so (20) makes every early A/B -job on time. By Table 3, each charge job is one unit late; summing their weights gives total charge contribution F_{sum} by (9). $\square$

Lemma 4 (One early job per incidence). *Every feasible schedule of cost at most $\overline{K}$ can be transformed in polynomial time into an integral feasible schedule of no greater cost with exactly one early member of each job pair (A_g, B_g) .*

Proof. Apply (1) and, if needed, compact the tail from H . The cost remains at most $\overline{K}$, Lemma 2 applies, and (21) holds. If every early count is one, we are done. Otherwise choose the largest index g for which $c_g \neq 1$. Higher counts are all one; (22) therefore forces $c_g = 0$, giving exactly one unit of slack in the suffix at g . Both members of this job pair are consequently in the tail. Thus A_g has a current tail slot; let C be its completion time in that slot. Let $e = e(g)$. Insert A_g into the early set, removing at most one job from a pair with a lower incidence rank as follows.

If no suffix with index below g is tight in (22), insertion alone preserves every inequality. Otherwise let $t < g$ be the largest tight suffix index. All suffixes with indices in $(t, g]$ have positive integer slack. The tight suffix at t contains $M - t + 1$ early jobs by (22); the suffix at g contains $M - g$, because $c_g = 0$ and $c_h = 1$ for $h > g$. Subtracting gives the number of early jobs with ranks in $[t, g - 1]$:

$$(M - t + 1) - (M - g) = g - t + 1 > 0.$$

Remove any such job D , with incidence rank $t \leq h < g$. After insertion and removal, suffix counts at indices at most h have net change zero, those between $h + 1$ and g increase by one but had slack, and all other suffixes are unchanged. Thus (22) remains valid, and every count remains in $\{0, 1, 2\}$.

Rebuild the early part using Lemma 3. Every rebuilt early A/B -job is on time, and all guards retain zero cost. If the total charge costs before and after rebuilding are Z_{old} and Z_{new} , respectively, then $0 \leq Z_{\text{old}}, Z_{\text{new}} \leq F_{\text{sum}}$, by Lemma 2 for the old charges, Lemma 3 for the rebuilt charges, and (9) for their total weight, so their increase is at most F_{sum} . Leave guards and all tail slots fixed, except the old slot of A_g , which completes at C . If D was removed, put it in that slot. Write its weight as $U + v_D$ with $1 \leq v_D \leq V$, by Table 3, (3), and (4). The common due-date formula in Table 3 gives $d_h = H - (h - 1)L$ and $d_g = H - (g - 1)L$. Since $h < g$, it follows that $d_h - d_g = (g - h)L \geq L$. The slot starts at or after H , and D is released before H by (19). Moreover, $C - d_g$ lies in $[P, (2M + 1)P]$ by (21). Also, $H + P \leq C \leq H + 2MP$ for this tail slot, and $0 \leq (h - 1)L \leq (M - 1)L = P - L < P$ by (12); hence $C - d_h = C - H + (h - 1)L$ also lies in $[P, (2M + 1)P]$. Thus replacement preserves feasibility, both costs are linear tardiness costs, and the tail saving is

$$\begin{aligned} & (U + \ell_e)(C - d_g) - (U + v_D)(C - d_h) \\ &= U(d_h - d_g) + \ell_e(C - d_g) - v_D(C - d_h) \\ &\geq UL - V(2M + 1)P > F_{\text{sum}}. \end{aligned}$$

The strict inequality is due to (14) of Lemma 1. The omitted term $\ell_e(C - d_g)$ is nonnegative, and the negative term is bounded using $v_D \leq V$ and the tail horizon. Consequently, writing J_{old} and J_{new} for the total schedule costs, we obtain

$$J_{\text{old}} - J_{\text{new}} \geq UL - V(2M + 1)P - (Z_{\text{new}} - Z_{\text{old}}) \geq Q(U + 1) > 0.$$

The last inequality uses $Z_{\text{new}} - Z_{\text{old}} \leq F_{\text{sum}}$ and the exact exchange margin (18). This bound covers simultaneous changes to all charges, not only to the two affected vertices.

If no job was removed, then after A_g has been moved into the early part, leave its old tail slot idle. By (21), deleting that tail slot saves at least $P(U + \ell_e)$, while rebuilding charges adds at most F_{sum} . Hence

$$J_{\text{old}} - J_{\text{new}} \geq P(U + \ell_e) - F_{\text{sum}} = Q \left[P_0(U + \ell_e) - \sum_{i=1}^n f_i \right] > 0,$$

The equality uses (9), and strict positivity follows from (15). In either case every job is still scheduled exactly once and total cost strictly decreases. Hence the schedule still has cost at most $\bar{K}$, and it remains integral because all jobs are placed in integer slots. It also continues to satisfy (21) for the following reason. The exchange only replaces jobs in existing tail slots or leaves such slots empty, so no occupied tail slot moves later than before. If a removed job D with incidence rank h is placed in such a slot, then $0 \leq (h-1)L \leq (M-1)L = P-L < P$ by (12), so the bound $P \leq C - d_h \leq (2M+1)P$ still applies. No tail sorting or compaction is required between exchanges. The largest defective rank has been corrected; every rank at least g now has count one, and any remaining or newly created defect has smaller rank. Repeating therefore terminates after at most M exchanges. Each exchange and slot reconstruction takes polynomial time and uses only the displayed job data and the current schedule. No optimality assumption is used. $\square$

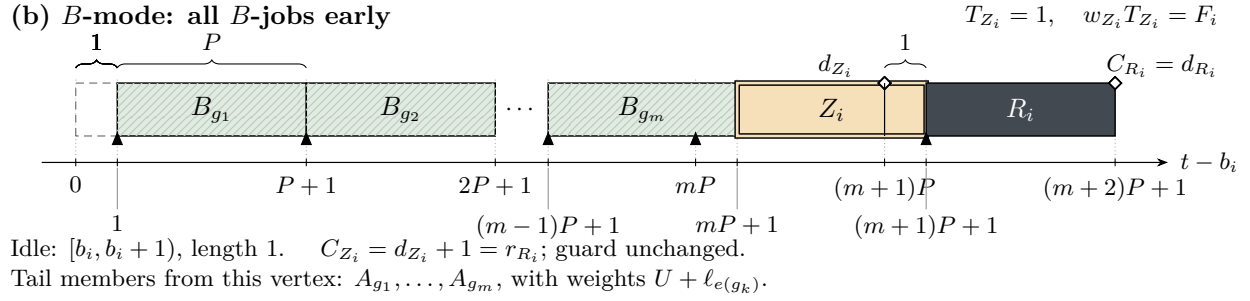
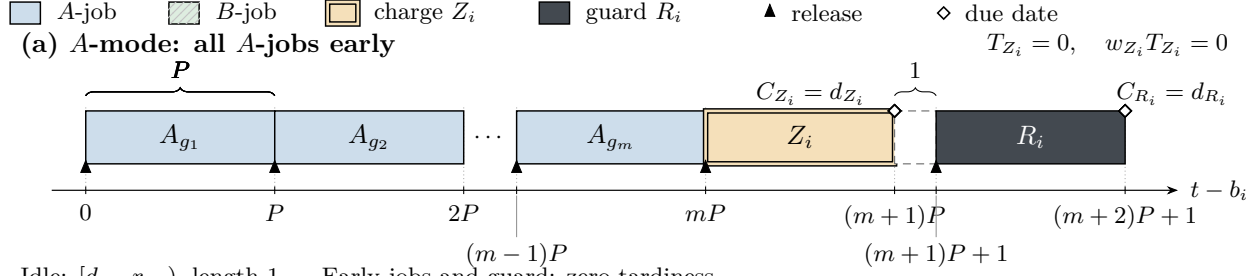
Lemma 5 (Vertex consistency). *Every integral feasible schedule of cost at most $\bar{K}$ with exactly one early member of each job pair (A_g, B_g) can be transformed in polynomial time, without increasing cost, into a schedule in which every vertex i uses one of two modes. In the A -mode, for every incidence g of i , job A_g is early and job B_g is in the tail; charge job Z_i completes at its due date and contributes zero to the objective. In the B -mode, every such B_g is early and the corresponding A_g is in the tail; Z_i completes one unit after its due date and contributes F_i . In either mode, guard job R_i starts at its release date.*

Figure 2 illustrates these two modes, including their release and due dates, the one-unit idle interval, and the complementary tail jobs.

Proof. There are M early A/B -jobs. Each of the n windows can fit at most m jobs by Lemma 2 and the window-capacity paragraph following it. Since $M = nm$, the M early jobs saturate all n windows, so every window contains exactly m jobs. Let $a_g = b_i + (k-1)P$ for $g = (i-1)m + k$, with $1 \leq i \leq n$ and $1 \leq k \leq m$. By the charge-start bound in Lemma 2, the job in local position k and its $m-k$ successors must finish by $b_i + mP + 1$. Its start s therefore satisfies $s \leq b_i + mP + 1 - (m-k+1)P = a_g + 1$. By (10) and Table 3, consecutive a_g values differ by P within a block and by $3P+1$ between blocks. Thus $a_{g+1} - a_g \geq P > 1$, using (12). Hence a job of rank $h > g$ is released at or after $a_h \geq a_g + P > a_g + 1$ and cannot occupy this position. If ρ_g is the incidence rank of the early A/B -job in chronological position g , it follows that $\rho_g \leq g$. Exactly one early member of every job pair is present, so $(\rho_1, \dots, \rho_M)$ is a permutation of $1, \dots, M$. Therefore $\sum_{g=1}^M (g - \rho_g) = 0$, while every summand is nonnegative, giving $\rho_g = g$ for every g . Each job pair contributes its early member in its own window and position; this conclusion did not assume that the initial early schedule had no idle time.

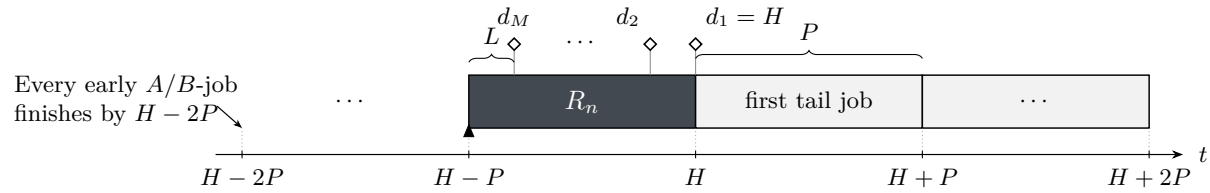
Left shift the resulting permutation using (1). Cost cannot increase; each guard remains at its release by Lemma 2. The order relative to the guards is unchanged, so the early A/B -jobs remain in their own windows. For $i > 1$, the job preceding the first such job of vertex i is guard R_{i-1} , which

Vertex block i : $g_k = (i - 1)m + k$, $1 \leq k \leq m$; every job has duration P .



(c) Global location of the A/B due dates and the tail

$$d_{A_g} = d_{B_g} = d_g = H - (g - 1)L, \quad P = ML, \quad g = 1, \dots, M.$$



$H - P < d_M \leq d_g \leq H$; every early A/B-job is on time.

Every tail job starts at or after H , so its completion time $C_g \geq H + P > d_g$ and its tardiness $T_g \geq P$.

Exact release and due dates

Job	Release date	Due date	Weight
A_{g_k}	$b_i + (k - 1)P$	$H - (g_k - 1)L$	$U + \ell_{e(g_k)}$
B_{g_k}	$b_i + (k - 1)P + 1$	$H - (g_k - 1)L$	$U + h_{e(g_k)}$
Z_i	$b_i + mP$	$b_i + (m + 1)P$	F_i
R_i	$b_i + (m + 1)P + 1$	$b_{i+1} = b_i + (m + 2)P + 1$	$\bar{K} + 1$

Scale convention. All processing rectangles in (a) and (b) have the same width, representing P ; the one-unit offsets are enlarged for visibility and the ellipses omit middle incidences. Panel (c) has a separate scale; the L spacing is schematic. All displayed time labels and formulas are exact.

Figure 2: The two normalized vertex modes in Lemma 5. In the A-mode the charge is on time and the one-unit idle interval precedes the guard. In the B-mode that idle interval is at the start of the block, all early jobs and the charge start one unit later, and the charge costs F_i . The guard has identical start and completion times in both modes, so no delay passes to the next block. Complementary jobs remain in the tail; they need not be consecutive there. Panel (c) shows why all early jobs are on time and all tail jobs are tardy. The tail is shown compacted from H .

completes at b_i ; for $i = 1$, the machine is initially available at $b_1 = 0$. Thus no delay is inherited from the preceding block. Write a local earliest start as $b_i + (k - 1)P + \varepsilon_k$. The recurrence is

$$\varepsilon_0 = 0, \quad \varepsilon_k = \max\{\varepsilon_{k-1}, z_k\} \quad (1 \leq k \leq m),$$

where $z_k = 0$ for a selected A and $z_k = 1$ for a selected B . Thus $\varepsilon_k = \max_{1 \leq h \leq k} z_h \in \{0, 1\}$. The final early completion is $b_i + mP + \varepsilon_m$, so

$$s_{Z_i} = b_i + mP + \varepsilon_m, \quad T_{Z_i} = \varepsilon_m.$$

Thus the charge contribution is $F_i \varepsilon_m$: it is zero when all selected jobs are A -jobs and equals F_i as soon as a B -job is selected. The guard following Z_i starts at

$$\max\{b_i + (m + 1)P + 1, b_i + (m + 1)P + \varepsilon_m\} = b_i + (m + 1)P + 1,$$

since $\varepsilon_m \leq 1$. It completes at b_{i+1} , which also verifies the block-boundary reset directly. The recurrence takes a maximum of the release offsets, so repeated B selections do not add further one-unit delays.

Thus the recurrence gives the two mode timings: in the A -mode, $\varepsilon_k = 0$ for every k , so one unit of idle time remains immediately before the guard; in the B -mode, $\varepsilon_k = 1$ for every k , so the charge job finishes exactly when the guard starts.

If a vertex has both A -jobs and B -jobs early, let $\mathcal{A}_i$ be the nonempty set of its incidence ranks whose A_g is early. For each $g \in \mathcal{A}_i$, move B_g from the tail into the early position of A_g , and move A_g to the old tail slot of B_g . Schedule the resulting early B -job at vertex i at the start $b_i + (k - 1)P + 1$, where $g = (i - 1)m + k$. Charge job Z_i already contributed F_i to total weighted tardiness and continues to do so. No other window or guard is affected. In each corresponding tail slot, replace B_g by A_g , retaining its completion time C_g . Both jobs have due date $d_g = H - (g - 1)L$, both are released before H , and $C_g \geq H + P > d_g$. The exact reduction in total cost is therefore

$$\sum_{g \in \mathcal{A}_i} [(U + h_{e(g)}) - (U + \ell_{e(g)})](C_g - d_g) = \sum_{g \in \mathcal{A}_i} \Delta_{e(g)}(C_g - d_g) \geq P \sum_{g \in \mathcal{A}_i} \Delta_{e(g)} > 0. \quad (24)$$

All charge-job contributions remain unchanged, and every replacement is within one job pair (A_g, B_g) with a common due date. Thus this strict saving follows directly from positive coefficients and positive tail tardiness; the due-date-gap margin used in Lemma 4 is unnecessary here. Apply this operation once to every vertex using both job types. The resulting schedule has the claimed modes, with all charge and guard placements and all releases respected. The transformation is polynomial and does not require optimality. $\square$

Call a schedule *canonical* if every vertex uses one of the two modes of Lemma 5, its charge starts immediately after its early A/B -jobs, every guard starts at its release, and all remaining A/B -jobs run consecutively from H in nonincreasing weight order. The early job at local position k of vertex

i starts at $b_i + (k - 1)P$ in the A -mode, or at $b_i + (k - 1)P + 1$ in the B -mode, for $1 \leq i \leq n$ and $1 \leq k \leq m$. Jobs in the tail with equal weights may be ordered arbitrarily.

Proposition 1 (Normalization of arbitrary bounded-cost schedules). *Every feasible schedule of cost at most $\overline{K}$, including one with noninteger starts, admits a canonical schedule of no greater cost. Given its job permutation, such a schedule can be constructed in polynomial time.*

Proof. Apply Lemma 4. This first left-shifts the input schedule, if necessary, and then produces an integral schedule of no greater cost with exactly one early member of each pair (A_g, B_g) . Next apply Lemma 5, which converts the schedule, again without increasing cost, so that every vertex uses one of the two modes and the charge and guard placements have the stated form.

It remains to arrange the tail. Compact the tail from H and sort it in nonincreasing weight order. Every tail job is already released and has due date at most H , so it remains tardy in every tail position. For the fixed tail set $\mathcal{T}$, its objective is

$$\sum_{j \in \mathcal{T}} w_j C_j - \sum_{j \in \mathcal{T}} w_j d_j.$$

The second term is independent of order. If adjacent jobs a, b have $w_a < w_b$, swapping them changes the weighted completion part by $P(w_a - w_b) < 0$. Ties change it by zero. Consequently nonincreasing weights minimize the tail cost; this is the equal-processing-time specialization of Smith's rule (Smith, 1956). Sorting changes neither feasibility nor any early contribution.

There are at most M exchanges for missing early members of the job pairs, at most n vertex-mode conversions, and one tail sort. All computations use only the input permutation and polynomial-bit data from the reduction; the possibly noninteger original start times need not be encoded or manipulated. $\square$

5 Cost identity and strong NP-completeness

Throughout this section, schedules are canonical in the sense of Proposition 1. For each vertex $i \in \mathcal{V}$, set $x_i = 1$ if vertex i uses the A -mode, so that all its A -jobs are early and the corresponding B -jobs are in the tail. Set $x_i = 0$ if vertex i uses the B -mode, so that all its B -jobs are early and the corresponding A -jobs are in the tail. Write $x = (x_i)_{i \in \mathcal{V}}$. Conversely, every vector $x \in \{0, 1\}^n$ defines a feasible canonical schedule: use the mode timings established in Lemma 5, then process the remaining A/B -jobs consecutively from H in nonincreasing weight order. All tail jobs are released before H by (19). For the fixed tail set, Proposition 1 shows that this order minimizes the tail contribution, with equal-weight ties irrelevant.

By Lemma 5, the charge-job contribution at vertex i is $F_i(1 - x_i)$. Early A/B -jobs and guards contribute zero by Lemma 2 and the early/tail separation established after its proof. Let $C(x)$ denote the total weighted tardiness of the canonical schedule associated with x ; the order of equal-weight tail jobs does not affect this value.

Proposition 2 (Cost identity for positive edge coefficients). *Let $n \geq 2$, and assign to every edge $e \in \mathcal{E}$ of the complete graph $(\mathcal{V}, \mathcal{E})$ a positive integer coefficient Δ_e . Using the coefficient array $(\Delta_e)_{e \in \mathcal{E}}$, define the job construction by (3)–(10) and Table 3, without imposing the graph-specific choice (2) or a source threshold. Recompute all derived parameters from this coefficient array, including U, G, Q, P, L, F_i , and $\bar{K}$. The normalization proposition remains valid, and the resulting scheduling instance has optimum*

$$\text{OPT} = \bar{K} - \frac{P}{2} \max_{x \in \{0,1\}^n} \sum_{1 \leq a < b \leq n} \Delta_{\{a,b\}} |x_a - x_b|.$$

Proof. Equations (3)–(4) give $1 \leq \ell_e < h_e \leq V$ and a one-unit gap between successive intervals. The definitions $q_e = M - 2t_e$ and $M = 2E$ give $q_e = 2(E - t_e) \geq 0$. Also $P_0 = 2M$ is even, $g_{i,e} - 1 \geq 0$, and each of the $n - 1$ incident coefficients is a positive integer and hence at least one. Thus (5) gives

$$f_i \geq \frac{3}{2} P_0 \sum_{e \in \mathcal{E}_i} \Delta_e \geq \frac{3}{2} P_0 (n - 1) > 0.$$

Thus $f_{\min} > 0$, so Q is well defined. The quantities U, G, Q, P, L, F_i , and $\bar{K}$ are positive integers. The quantities q_e , block boundaries, and release dates are nonnegative integers in the ranges already specified. With $\delta = 2P_0 f_{\min} Q - G$, the identities established in Lemma 1 give the following strict margins.

$$\begin{aligned} P &= ML > 1, \\ \left(\min_{1 \leq i \leq n} F_i \right) (2P + 1) - \bar{K} &= Q(\delta + f_{\min}) > 0, \\ UL - V(2M + 1)P - F_{\text{sum}} &= Q(U + 1) > 0, \\ PU - F_{\text{sum}} &= Q \left(P_0 U - \sum_{i=1}^n f_i \right) > 0. \end{aligned}$$

Here $1 \leq \delta \leq 2P_0 f_{\min}$ follows from the floor definition of the recomputed Q . This is (8); see the calculation of δ in the proof of Lemma 1. The charge and exchange equalities above are (17) and (18), reapplied to the recomputed parameters. No graph-specific coefficient bound occurs in these identities.

The release and due-date identities (19) and (20) depend only on $P = ML$ and the block definitions. The guard weight remains $\bar{K} + 1$. Thus Lemma 2 applies, compaction gives (21), and Lemmas 3, 4, and 5 remain valid: they use positive interval endpoints, the release/due-date identities, and inequalities (12)–(15), all re-established above for the new coefficients. In particular, Lemma 5 uses only the strict positivity of each Δ_e . The reference schedule in which every vertex uses the A -mode still has cost $\bar{K}$, by its construction and evaluation in Section 3.2, using (7) and (9). An optimum exists by the finite earliest-permutation argument following (1); the reference

schedule places its cost within the normalization bound. Proposition 1 therefore applies to the supplied coefficient array.

Fix $e = \{a, b\} \in \mathcal{E}$ with $a < b$ and abbreviate $g_a = g_{a,e}$, $g_b = g_{b,e}$, $q = q_e$, $\Delta = \Delta_e$, and $u = U + \ell_e$. The two tail weights are $u + \Delta x_a$ and $u + \Delta x_b$. This follows from the mode definition at the start of this section, the weights in Table 3, and $h_e = \ell_e + \Delta_e$ in (3). By the canonical weight order in Proposition 1 and the disjoint ordered intervals in (3), the $E - t_e$ higher intervals precede these two jobs. Each higher interval contains exactly two tail jobs by Lemma 5; hence precisely $2(E - t_e) = q_e = q$ tail jobs precede them, independently of the bits. The two completions are $H + (q + 1)P$ and $H + (q + 2)P$. Table 3 gives $d_{g_a} = H - (g_a - 1)L$ and $d_{g_b} = H - (g_b - 1)L$, and the larger weight goes first by Proposition 1. Their combined tail cost is

$$\begin{aligned} & P[(q + 1) \max\{u + \Delta x_a, u + \Delta x_b\} \\ & \quad + (q + 2) \min\{u + \Delta x_a, u + \Delta x_b\}] \\ & \quad + L[(g_a - 1)(u + \Delta x_a) + (g_b - 1)(u + \Delta x_b)]. \end{aligned} \tag{25}$$

By (5) and the scaling definitions (9),

$$F_v = \sum_{e \in \mathcal{E}_v} \Delta_e \left[P \left(q_e + \frac{3}{2} \right) + L(g_{v,e} - 1) \right].$$

Lemma 5 multiplies each endpoint's summand by $1 - x_v$. We therefore allocate to edge e the following portions of the charge contributions of Z_a and Z_b :

$$\begin{aligned} & \Delta \left[P \left(q + \frac{3}{2} \right) + L(g_a - 1) \right] (1 - x_a) \\ & \quad + \Delta \left[P \left(q + \frac{3}{2} \right) + L(g_b - 1) \right] (1 - x_b). \end{aligned} \tag{26}$$

Define

$$\begin{aligned} W_e &= P(2q + 3) + L(g_a + g_b - 2), \\ D_e &= (u + \Delta)W_e \\ &= (U + h_e)[P(2q_e + 3) + L(g_{a,e} + g_{b,e} - 2)], \end{aligned} \tag{27}$$

and define $\Phi_e(x_a, x_b)$ as the sum of the tail contribution (25) and the allocated charge contribution (26). To expose the cancellation, put $\sigma_e = x_a + x_b$ and $\eta_e = |x_a - x_b|$. Since the modes are binary,

$$\max\{x_a, x_b\} = \frac{\sigma_e + \eta_e}{2}, \quad \min\{x_a, x_b\} = \frac{\sigma_e - \eta_e}{2}.$$

Substituting these binary max/min identities into (25), and using (27), gives the tail contribution

$$uW_e + \frac{P\Delta}{2} [(2q + 3)\sigma_e - \eta_e] + L\Delta[(g_a - 1)x_a + (g_b - 1)x_b].$$

Expanding (26) with the same definition of W_e gives the allocated charge contribution

$$\Delta W_e - \frac{P\Delta}{2}(2q+3)\sigma_e - L\Delta[(g_a-1)x_a + (g_b-1)x_b].$$

Thus the linear mode terms, including their incidence-specific due-date offsets, cancel exactly, and

$$\begin{aligned}\Phi_e(x_a, x_b) &= (u + \Delta)W_e - \frac{P\Delta}{2}\eta_e \\ &= D_e - \frac{P\Delta_e}{2}|x_a - x_b|.\end{aligned}\tag{28}$$

The charge coefficients therefore leave only a constant and the saving for unequal endpoint modes.

In the equal-weight cases, either order gives the same combined cost even when $g_a \neq g_b$, because the weighted due-date sum is unchanged. As in the fixed-tail interchange argument in Proposition 1, when $w_a = w_b$, both $w_a C_a + w_b C_b$ and $w_a d_a + w_b d_b$ are unchanged by interchanging the jobs. Table 4 records the resulting four values.

Table 4: Edge contribution after normalization; values follow from (28).

x_a	x_b	Edge contribution
0	0	D_e
0	1	$D_e - P\Delta_e/2$
1	0	$D_e - P\Delta_e/2$
1	1	D_e

Every charge-job contribution has been allocated exactly once: for each vertex v , summing its endpoint allocations over $e \in \mathcal{E}_v$ gives $Qf_v(1 - x_v) = F_v(1 - x_v)$. Indeed, (26) includes one summand for each incident edge; summing these at v reproduces (5) scaled by (9). Early A/B -jobs and guards have zero cost, and $\sum_{e \in \mathcal{E}} D_e = QG = \bar{K}$ by (27), (7), and (9); the factor Q enters through both $P = QP_0$ and $L = QL_0$. Summing (28) over all $e \in \mathcal{E}$ now gives the complete schedule cost: Lemma 5 and the early/tail separation leave no additional nonzero contributions.

$$C(x) = \bar{K} - \frac{P}{2} \sum_{1 \leq a < b \leq n} \Delta_{\{a,b\}} |x_a - x_b|. \tag{29}$$

Every bit vector defines a feasible canonical schedule by the mode-and-tail construction at the start of Section 5. An optimum has cost at most $\bar{K}$ by the reference schedule in Section 3.2, and hence can be normalized by Proposition 1. Thus the optimum is the minimum of (29) over $x \in \{0, 1\}^n$, which is

$$\bar{K} - \frac{P}{2} \max_{x \in \{0,1\}^n} \sum_{1 \leq a < b \leq n} \Delta_{\{a,b\}} |x_a - x_b|.$$

This proves the proposition. □

Proposition 2 requires a strictly positive integer coefficient on every edge of the auxiliary complete graph. It does not apply, as stated, to zero, negative, or arbitrary real coefficients. Each coefficient Δ_e is a construction parameter: by (3) it determines the interval width $h_e - \ell_e$; by Table 3 it determines the weight difference between the A - and B -jobs for both endpoint incidences; and by (5) and (9) it determines the corresponding portions of the endpoint charge weights.

We now return to the specific coefficient choice in (2): original graph edges have coefficient $\Gamma + 1$, while added auxiliary edges have coefficient 1.

Theorem 1. *The decision version of $1 \mid r_j, p_j = p \mid \sum_j w_j T_j$ is strongly NP-complete, even under $d_j \geq r_j + p$ for every job. In the reduction, all numerical data, including the decision threshold, are bounded by a polynomial in the number of jobs.*

Proof. Correctness of the reduction. Let $c_G(x) = \sum_{\substack{\{a,b\} \in \mathcal{F} \\ a < b}} |x_a - x_b|$ be the number of original edges cut by x , and let $s(x) = \sum_{i=1}^n x_i$ be the size of one side. Substituting (2), the coefficient 1 assigned to every auxiliary complete-graph edge contributes $s(x)(n - s(x))$, the number of crossing pairs in the complete graph. The additional Γ assigned to original graph edges contributes $\Gamma c_G(x)$. The upper bound $s(x)(n - s(x)) \leq \lfloor n^2/4 \rfloor = \Gamma - 1$ follows from the definition of Γ in (2). Thus

$$\sum_{1 \leq a < b \leq n} \Delta_{\{a,b\}} |x_a - x_b| = \Gamma c_G(x) + s(x)(n - s(x)), \quad 0 \leq s(x)(n - s(x)) \leq \Gamma - 1. \quad (30)$$

Combining (11), (29), and (30) gives

$$C(x) - K_* = \frac{P}{2} [\Gamma(\kappa - c_G(x)) - s(x)(n - s(x))]. \quad (31)$$

For the forward direction, suppose the MAX-CUT instance is YES and choose x with $c_G(x) \geq \kappa$. The mode timings of Lemma 5, followed by the tail construction at the start of Section 5, give a feasible canonical schedule. Equation (31) gives $C(x) \leq K_*$, since $\kappa - c_G(x) \leq 0$ and $s(x)(n - s(x)) \geq 0$.

For the reverse direction, suppose the constructed scheduling instance has a feasible schedule with cost at most K_* . By (11),

$$K_* = \overline{K} - \frac{P}{2} \Gamma \kappa \leq \overline{K},$$

since $P > 0$, $\Gamma > 0$, and $\kappa \geq 1$. Thus the schedule satisfies the cost bound required by Proposition 1, which gives a canonical schedule of no greater cost. If x denotes its mode vector, then $C(x) \leq K_*$. If its cut had $c_G(x) \leq \kappa - 1$, then (31), together with $s(x)(n - s(x)) \leq \Gamma - 1$ from (30), would imply

$$C(x) - K_* \geq \frac{P}{2} [\Gamma - (\Gamma - 1)] = \frac{P}{2} > 0,$$

a contradiction. Hence $c_G(x) \geq \kappa$. Therefore, the MAX-CUT instance is YES if and only if the constructed scheduling instance admits a schedule of cost at most K_* .

Validity and polynomial bounds for the construction. The threshold is integral because P is even. Indeed, $P = QP_0$ by (9) and $P_0 = 2M$; Γ and κ are integers as well, so (11) is integral. For

nonnegativity, (2) and (4) give $V = 2E + \Gamma|\mathcal{F}| \geq \Gamma\kappa$, since $\kappa \leq |\mathcal{F}|$. Equation (6) gives $U > V$. Hence

$$\Gamma\kappa \leq \Gamma|\mathcal{F}| \leq V < U.$$

In the reference schedule from Section 3.2, every vertex uses the A -mode. The M jobs processed after H have completion offsets tP , for $1 \leq t \leq M$; each of these jobs has weight at least U , and each has due date at most H by Table 3. Thus $\bar{K} \geq UP \sum_{t=1}^M t = UPM(M+1)/2$. Since $M \geq 2$, this exceeds $P\Gamma\kappa/2$, so $K_* > 0$. Equations (19) and (20), together with Table 3, show that every constructed job satisfies $d_j \geq r_j + P$.

The strict inequalities needed for schedule normalization were proved in Lemma 1; here only polynomial magnitude is needed. By (2) and (4), using $E = \binom{n}{2}$, $M = n(n-1)$ from Section 3.1, and $\Gamma = O(n^2)$ from (2),

$$V = E + \sum_{e \in \mathcal{E}} \Delta_e = 2E + \Gamma|\mathcal{F}| \leq (2 + \Gamma) \binom{n}{2} = O(n^4), \quad M = O(n^2).$$

Every q_e and incidence rank is $O(M)$, $P_0 = 2M$, and $L_0 = 2$. Thus the coefficient of Δ_e in (5) is $O(M^2)$. Each edge occurs twice in the incidence sum, and (4) gives

$$\sum_{i=1}^n \sum_{e \in \mathcal{E}_i} \Delta_e = 2 \sum_{e \in \mathcal{E}} \Delta_e \leq 2V.$$

Consequently (5) gives the first bound below, and (6) gives the second:

$$\sum_{i=1}^n f_i = O(VM^2), \quad U = O(VM^2).$$

Each bracket in (7) is $O(M^2)$, $h_e \leq V$ by (3)–(4), and there are $E = M/2$ edges by Section 3.1. Thus (7) gives $G = O((U + V)M^3)$. Equation (8) and positivity of the integers $P_0, f_{\min}$ give

$$1 \leq Q = \left\lfloor \frac{G}{2P_0 f_{\min}} \right\rfloor + 1 \leq G + 1.$$

Table 5 collects explicit degree estimates; the hidden constants are uniform and independent of the source graph.

To verify the scaling rows of Table 5, the preceding estimate gives $G = O((U + V)M^3) = O(n^{14})$. Equation (8) gives $Q \leq G + 1 = O(n^{14})$. Since $P_0 = 2M$ and $L_0 = 2$, (9) gives $L = 2Q = O(n^{14})$ and $P = 2MQ = O(n^{16})$. Next, (10) gives $H = n((n+1)P + 1) = O(n^{18})$, and Table 3 bounds all release and due dates by H , giving the same exponent. Equation (9) gives $F_i = Qf_i = O(n^{22})$ and $\bar{K} = QG = O(n^{28})$. The threshold argument above gives $0 \leq K_* \leq \bar{K}$, so $K_* = O(n^{28})$.

The largest weights are the guards' $\bar{K} + 1$ from Table 3. Indeed, (6), (7), and (9) give $F_i = Qf_i < QU < QG = \bar{K}$. Also $U + h_e \leq U + V < G \leq QG$: for the last edge, $h_e = V$ and $q_e = 0$, so its summand in (7) alone exceeds $U + V$. Thus all nonguard weights are below $\bar{K} + 1$. Therefore,

Table 5: Polynomial magnitude bounds in the number of retained graph vertices

Quantity	Magnitude bound in n
$\Gamma, \Delta_e, M, P_0, q_e, g_{i,e}$	$O(n^2)$
V	$O(n^4)$
$\sum_{i=1}^n f_i, U$	$O(n^8)$
G, Q, L	$O(n^{14})$
P	$O(n^{16})$
H, r_j, d_j	$O(n^{18})$
F_i	$O(n^{22})$
$\bar{K}, K_*, w_j$	$O(n^{28})$

every target integer has $O(\log n)$ bits and magnitude $O(n^{28})$. The job construction in Section 3.1 gives $N = 2M + 2n = 2n^2$ jobs: two for each incidence and a charge and guard for each vertex. Hence $O(n^{28}) = O(N^{14})$. There are $O(N)$ numerical job entries, each of unary length $O(N^{14})$, and one threshold, giving total unary encoding length $O(N^{15})$. Since the source graph has no isolated vertices, $n \leq 2|\mathcal{F}|$, so this unary output length is polynomial in the graph input length. The reduction therefore proves strong NP-hardness.

All parameters can be computed with $O(n^2)$ arithmetic operations, using prefix sums for (3) and direct sums for (5) and (7). Their operands have polynomial bit length, so standard integer arithmetic, including the division in (8), takes polynomial time. The coefficients Δ_e specify interval endpoints, not numbers of jobs or time-indexed objects to enumerate.

Membership in NP. For an arbitrary input instance with N jobs and common processing time p , a job permutation is a certificate using $O(N \log N)$ bits. Compute its earliest schedule by (1). Writing $r_{\max} = \max_j r_j$ and $w_{\max} = \max_j w_j$, induction in (1) gives $\hat{C}_k \leq r_{\max} + kp$, so every computed completion time is at most $r_{\max} + Np$, and total weighted tardiness is at most

$$Nw_{\max}(r_{\max} + Np + \max_j |d_j|).$$

The certificate and these computations have polynomial bit length. If any feasible schedule with the given permutation meets the decision threshold, its earliest schedule does too, including when original start times are noninteger, by the dominance argument immediately following (1). Thus the decision problem belongs to NP. Together with the reduction above, this proves strong NP-completeness. $\square$

Remark (Interpretation of the optimum). Retain the notation from the proof of Theorem 1: $c_G(x)$ is the number of original graph edges cut by x , and $s(x) = \sum_{i=1}^n x_i$ is the size of one side of the cut. Proposition 2, specialized using (30), gives

$$\text{OPT} = \bar{K} - \frac{P}{2} \max_{x \in \{0,1\}^n} [\Gamma c_G(x) + s(x)(n - s(x))]. \quad (32)$$

Thus minimizing the scheduling objective first maximizes the number of original graph edges cut and, among maximum cuts, maximizes $s(x)(n - s(x))$. Indeed, by (30), $0 \leq s(x)(n - s(x)) \leq \Gamma - 1$. Losing one original cut edge therefore reduces the first term in the maximized expression by Γ , while the largest possible increase in the second term is only $\Gamma - 1$.

6 A phase-grid assignment algorithm and its shifted-objective guarantee

This section complements the strong NP-hardness result with a polynomial-time phase-grid assignment algorithm and a worst-case analysis. The analysis applies to arbitrary instances of $1 \mid r_j, p_j = p \mid \sum_j w_j T_j$ and is independent of the reduction in the preceding sections.

6.1 Shifted objective

For a feasible schedule S , write

$$\begin{aligned} F(S) &= \sum_j w_j (C_j - d_j)^+, \\ W &= \sum_j w_j, \\ F^* &= \min_S F(S). \end{aligned}$$

We analyze the schedule-independent shift

$$\begin{aligned} B(I) &= pW, \\ \Phi(S) &= F(S) + B(I) = \sum_j w_j (T_j(S) + p). \end{aligned} \tag{33}$$

Thus $\Phi^* = F^* + pW > 0$, even when all jobs can be completed on time. The shift scales naturally under changes of time or weight units, does not depend on the algorithm's output, and is independent of the schedule. Hence F and Φ have the same ordering and the same minimizers. The shift does, however, change approximation ratios, so the guarantee below is explicitly a guarantee for Φ , not for the unshifted objective F .

6.2 The phase-grid assignment algorithm

For $a \in [0, p)$, the phase- a grid consists of starts $a + kp$, $k \in \mathbb{Z}$, subject to nonnegative starting times. Let

$$\mathcal{A} = \{r_j \bmod p : 1 \leq j \leq N\}.$$

For each distinct phase $a \in \mathcal{A}$, the algorithm performs the following steps.

1. Round every release date upward to the phase- a grid:

$$R_j(a) = a + p \left\lceil \frac{r_j - a}{p} \right\rceil.$$

2. Generate the candidate starts

$$\mathcal{T}_a = \{R_i(a) + kp : 1 \leq i \leq N, 0 \leq k < N\}.$$

3. Construct a bipartite graph with one node for every job and one node for every distinct $t \in \mathcal{T}_a$. Include edge (j, t) if $t \geq r_j$, with cost $w_j(t + p - d_j)^+$, and find a minimum-cost matching covering every job.
4. Interpret the matching as a schedule and retain the least-cost schedule over all phases.

Different slots on one grid define disjoint intervals of length p , so every matching is feasible. Unmatched slots represent idle time; the matching must cover the jobs but need not cover all slots.

Lemma 6. *For a fixed phase a , the matching computes an optimal schedule among all schedules whose starts lie on the phase- a grid.*

Proof. Take any feasible grid schedule and preserve its job order. Left justify it within the grid: start the first job at its rounded release and each successor at the maximum of its rounded release and the preceding completion time. This cannot increase any completion time. Every maximal consecutive block begins at some $R_i(a)$, and successive jobs in that block start at $R_i(a) + kp$ for $0 \leq k < N$. Hence all starts belong to $\mathcal{T}_a$. Conversely, any matching covering all jobs gives a feasible grid schedule whose objective value is exactly its total edge cost. $\square$

There are at most N phases, at most N^2 slots per phase, and at most N^3 assignment edges per phase. A rectangular Hungarian implementation uses $O(N^2M)$ arithmetic operations for N jobs and $M = |\mathcal{T}_a|$ slots, so the full algorithm uses $O(N^5)$ arithmetic operations. All generated times and costs have polynomial binary length. In particular, the algorithm does not enumerate all integer times, every residue $0, \dots, p-1$, or the full time horizon.

6.3 Worst-case analysis

For any job order π , left shifting produces its earliest feasible schedule:

$$s_{\pi(1)} = r_{\pi(1)}, \quad s_{\pi(k)} = \max\{r_{\pi(k)}, s_{\pi(k-1)} + p\} \quad (k \geq 2).$$

It follows that an optimum exists in which every start has the form $r_i + kp$ for some $0 \leq k < N$. Consequently, we can choose an earliest optimal schedule in which the residue of every start belongs to $\mathcal{A}$.

Lemma 7. *If all starts in a feasible schedule are rounded upward to the same phase- a grid, the resulting schedule is feasible. If job j is delayed by $\delta_j(a)$, then $0 \leq \delta_j(a) < p$ and its cost increases by at most $w_j \delta_j(a)$.*

Proof. For each job j , let

$$\hat{s}_j = a + p \left\lceil \frac{s_j - a}{p} \right\rceil$$

denote its start time after upward rounding to the phase- a grid, and let

$$\delta_j(a) = \hat{s}_j - s_j.$$

Since $x \leq \lceil x \rceil < x + 1$, we have $0 \leq \delta_j(a) < p$. Furthermore, $\hat{s}_j \geq s_j \geq r_j$, so all release-date constraints remain satisfied. Consider two successive jobs j and k in the original schedule. Feasibility of that schedule implies $s_k \geq s_j + p$. Therefore,

$$\begin{aligned} \hat{s}_k &= a + p \left\lceil \frac{s_k - a}{p} \right\rceil \\ &\geq a + p \left\lceil \frac{s_j + p - a}{p} \right\rceil \\ &= a + p \left\lceil \frac{s_j - a}{p} \right\rceil + p \\ &= \hat{s}_j + p. \end{aligned}$$

Thus, the rounded processing intervals remain disjoint, and the resulting schedule is feasible. The rounded completion time of job j is $\hat{C}_j = C_j + \delta_j(a)$. Hence the weighted-tardiness cost of job j increases by at most $w_j \delta_j(a)$. $\square$

Uniformly averaging the grid phase over $[0, p)$ gives mean displacement $p/2$ for each fixed start and hence the additive bound $pW/2$. Weight-dependent averaging over the phases actually enumerated by the algorithm yields the sharper result below.

Theorem 2. *Let S_{alg} be the schedule returned by the phase-grid assignment algorithm, and define*

$$F_{\text{alg}} = F(S_{\text{alg}}), \quad \Phi_{\text{alg}} = \Phi(S_{\text{alg}}) = F_{\text{alg}} + pW.$$

Then

$$F_{\text{alg}} \leq F^* + D(I), \quad D(I) = \frac{p}{2W} \left(W^2 - \sum_j w_j^2 \right). \quad (34)$$

Consequently,

$$\frac{\Phi_{\text{alg}}}{\Phi^*} \leq \frac{3}{2} - \frac{\sum_j w_j^2}{2W^2} \leq \frac{3}{2} - \frac{1}{2N} < \frac{3}{2}. \quad (35)$$

Proof. Fix an earliest optimal schedule and let $a_i = s_i^* \bmod p$. Every a_i is tested. For each i , round all optimal starts upward to the phase- a_i grid, and let $\delta_j(a_i)$ be the resulting delay of job j . By Lemmas 6 and 7,

$$F_{\text{alg}} \leq F^* + \min_i \sum_j w_j \delta_j(a_i).$$

Choose index i with probability w_i/W for the analysis only. Then

$$\min_i \sum_j w_j \delta_j(a_i) \leq \frac{1}{W} \sum_i \sum_j w_i w_j \delta_j(a_i).$$

When $a_i = a_j$, both cross-displacements are zero. Otherwise, the two forward distances around a circle of circumference p satisfy

$$\delta_j(a_i) + \delta_i(a_j) = p.$$

The diagonal terms vanish. Grouping the double sum into unordered pairs gives

$$\frac{1}{W} \sum_i \sum_j w_i w_j \delta_j(a_i) = \frac{p}{W} \sum_{\substack{i < j \\ a_i \neq a_j}} w_i w_j \leq \frac{p}{W} \sum_{i < j} w_i w_j = D(I),$$

which proves (34). Adding the shift to both the algorithmic and optimal values and using $\Phi^* \geq pW$ yields

$$\frac{\Phi_{\text{alg}}}{\Phi^*} \leq 1 + \frac{D(I)}{F^* + pW} \leq 1 + \frac{D(I)}{pW} = \frac{3}{2} - \frac{\sum_j w_j^2}{2W^2}.$$

Finally, $\sum_j w_j^2 \geq W^2/N$ by Cauchy–Schwarz. The algorithm itself is deterministic; the random choice is used only in the proof. $\square$

For $N = 1$, $D(I) = 0$ and the algorithm is exact. More generally, let q^* be the number of distinct phases among the optimal starts used in the proof, and let $q = |\mathcal{A}|$; then $q^* \leq q$. If W_h is the total weight of the jobs in optimal-start phase h , grouping the pair terms by phase and applying Cauchy–Schwarz gives

$$F_{\text{alg}} - F^* \leq \frac{p}{2W} \left(W^2 - \sum_{h=1}^{q^*} W_h^2 \right) \leq \frac{pW}{2} \left(1 - \frac{1}{q^*} \right) \leq \frac{pW}{2} \left(1 - \frac{1}{q} \right).$$

Hence the ratio is also at most $3/2 - 1/(2q)$. When all releases share one phase, the algorithm is exact.

6.4 Tightness and the role of the shift

Proposition 3. *For every integer $N \geq 1$, an integer-data instance attains ratio $3/2 - 1/(2N)$ for the phase-grid assignment algorithm and the shift (33).*

Proof. Set $p = N$. For $i = 0, \dots, N - 1$, create a unit-weight job with

$$r_i = i(2N + 1), \quad d_i = r_i + N.$$

Executing each job at its release is feasible and has zero cost. Thus $F^* = 0$, $W = N$, and $\Phi^* = N^2$. The release residues are $0, 1, \dots, N - 1$. At any tested phase, rounding the releases produces a permutation of the delays $0, 1, \dots, N - 1$. The rounded intervals remain disjoint because consecutive releases are $2N + 1$ apart and each delay is at most $N - 1$. On a fixed tested phase, job i cannot start before its rounded release date and therefore incurs tardiness at least equal to its rounding delay. Scheduling every job at its rounded release date is feasible, so the optimum on each tested grid is the sum of the delays. Hence $F_{\text{alg}} = N(N - 1)/2$, and

$$\frac{\Phi_{\text{alg}}}{\Phi^*} = \frac{N^2 + N(N - 1)/2}{N^2} = \frac{3}{2} - \frac{1}{2N}.$$

□

This proves tightness for the specified phase-grid output, not approximation hardness for the shifted problem. Left justifying the algorithm's final job order in unrestricted continuous time can only improve its value, but it eliminates the loss on this particular family.

For any prescribed shift $B(I) > 0$, the same additive analysis gives

$$\frac{F_{\text{alg}} + B(I)}{F^* + B(I)} \leq 1 + \frac{D(I)}{F^* + B(I)} \leq 1 + \frac{pW}{2B(I)}. \quad (36)$$

Thus $B(I) = \lambda pW$ for fixed $\lambda > 0$ gives ratio at most $1 + 1/(2\lambda)$. Merely increasing the shift can improve this displayed ratio without improving the schedule, so such a change is not a PTAS for a fixed objective.

7 Conclusion

We have shown that the single-machine total weighted tardiness problem with release dates remains strongly NP-hard when every job has the same processing time and is individually capable of meeting its due date (Theorem 1). This settles the complexity question documented by van den Akker et al. (2010) and Gafarov et al. (2020). Complementing the hardness result, we developed a deterministic phase-grid assignment algorithm that solves at most N minimum-cost assignment problems and runs in $O(N^5)$ arithmetic operations. For the shifted objective $\Phi = F + p \sum_j w_j$, its approximation ratio is at most $3/2 - 1/(2N)$, and this bound is tight for the specified algorithm (Theorem 2 and Proposition 3).

The normalization argument (Proposition 1) is the main structural step in the hardness proof. It applies to every sufficiently inexpensive feasible schedule and produces one binary choice per vertex, after which an exact pairwise cost identity (Proposition 2 and (29)) gives the MAX-CUT

correspondence. All job counts and numerical values remain polynomially bounded (Table 5). For the algorithmic result, the key step is instead to average the rounding losses over the release phases with weight-dependent probabilities. This yields the sharper additive error (34), from which the shifted ratio follows. Because an objective shift changes approximation ratios, this result is not a multiplicative guarantee for ordinary total weighted tardiness.

The established polynomial cases in the literature remain useful directions for algorithm design. Further restrictions on release dates, weights, their ordering, or the number of release phases may support stronger algorithms even though the general problem is strongly NP-hard. Determining whether the unshifted problem admits a meaningful multiplicative guarantee on restricted instance classes, and whether other polynomial algorithms improve on the phase-grid assignment bound for the shifted objective, remain questions for future work.

Data and code availability

No external empirical data are used. The proofs of Theorems 1 and 2 are self-contained and do not depend on computational experiments. The present manuscript package contains the L^AT_EX source with an internal bibliography and its compiled PDF; it does not include supplementary implementation code or recorded computational outputs.

Acknowledgments

Generative artificial-intelligence tools assisted with proof development, computational checks, literature searches, and manuscript drafting.

References

- Aktürk, M.S., D. Özdemir. (2000). An exact approach to minimizing total weighted tardiness with release dates. *IIE Transactions*, 32, 1091–1101. <https://doi.org/10.1023/A:1013741325877>.
- Aktürk, M.S., D. Özdemir. (2001). A new dominance rule to minimize total weighted tardiness with unequal release dates. *European Journal of Operational Research*, 135(2), 394–412.
- Baptiste, P. (1999). Polynomial time algorithms for minimizing the weighted number of late jobs on a single machine with equal processing times. *Journal of Scheduling*, 2(6), 245–252.
- Baptiste, P. (2000). Scheduling equal-length jobs on identical parallel machines. *Discrete Applied Mathematics*, 103(1–3), 21–32. [https://doi.org/10.1016/S0166-218X\(99\)00238-3](https://doi.org/10.1016/S0166-218X(99)00238-3).
- Baptiste, P., P. Brucker, S. Knust, V.G. Timkovsky. (2004). Ten notes on equal-processing-time scheduling: At the frontiers of solvability in polynomial time. *4OR*, 2(2), 111–127. <https://doi.org/10.1007/s10288-003-0024-4>.

- Brucker, P., S.A. Kravchenko. (2005). *Scheduling jobs with release times on parallel machines to minimize total tardiness*. Universität Osnabrück. Fachbereich Mathematik/Informatik
- Brucker, P., S.A. Kravchenko. (2006). Scheduling equal processing time jobs to minimize the weighted number of late jobs. *Journal of Mathematical Modelling and Algorithms*, 5(2), 143–165. <https://doi.org/10.1007/s10852-005-9011-4>.
- Carlier, J. (1981). Problèmes d’ordonnancement à durées égales. *QUESTIO*, 5(4), 219–228.
- Chrobak, M., C. Dürr, W. Jawor, Ł. Kowalik, M. Kurowski. (2006). A note on scheduling equal-length jobs to maximize throughput. *Journal of Scheduling*, 9(1), 71–73. <https://arxiv.org/abs/cs/0410046>.
- Dessouky, M.I., B.J. Lageweg, J.K. Lenstra, S.L. van de Velde. (1990). Scheduling identical jobs on uniform parallel machines. *Statistica Neerlandica*, 44(3), 115–123. <https://doi.org/10.1111/j.1467-9574.1990.tb01276.x>.
- Du, J., J.Y.-T. Leung. (1990). Minimizing total tardiness on one machine is NP-hard. *Mathematics of Operations Research*, 15(3), 483–495. <https://doi.org/10.1287/moor.15.3.483>.
- Gafarov, E.R., A.A. Lazarev, F. Werner. (2020). Minimizing total weighted tardiness for scheduling equal-length jobs on a single machine. *Automation and Remote Control*, 81(5), 853–868. <https://doi.org/10.1134/S0005117920050069>.
- Garey, M.R., D.S. Johnson, L. Stockmeyer. (1976). Some simplified NP-complete graph problems. *Theoretical Computer Science*, 1(3), 237–267. [https://doi.org/10.1016/0304-3975\(76\)90059-1](https://doi.org/10.1016/0304-3975(76)90059-1).
- Graham, R.L., E.L. Lawler, J.K. Lenstra, A.H.G. Rinnooy Kan. (1979). Optimization and approximation in deterministic sequencing and scheduling: A survey. *Annals of Discrete Mathematics*, 5, 287–326. [https://doi.org/10.1016/S0167-5060\(08\)70356-X](https://doi.org/10.1016/S0167-5060(08)70356-X).
- Heeger, K., H. Molter. (2025). Minimizing the number of tardy jobs with uniform processing times on parallel machines. In *42nd International Symposium on Theoretical Aspects of Computer Science, Leibniz International Proceedings in Informatics* (Vol. 327, pp. 47:1–47:17). Schloss Dagstuhl–Leibniz-Zentrum für Informatik. <https://doi.org/10.4230/LIPIcs.STACS.2025.47>.
- Kolliopoulos, S.G., G. Steiner. (2006). Approximation algorithms for scheduling problems with a modified total weighted tardiness objective. Manuscript submitted to Elsevier, December 8, 2006. <https://cgi.di.uoa.gr/~sgk/papers/web-tard-mod.pdf>.
- Kovalyov, M.Y., F. Werner. (2002). Approximation schemes for scheduling jobs with common due date on parallel machines to minimize total tardiness. *Journal of Heuristics*, 8, 415–428. <https://doi.org/10.1023/A:1015487829051>.
- Kravchenko, S.A., F. Werner. (2011). Parallel machine problems with equal processing times: A survey. *Journal of Scheduling*, 14(5), 435–444. <https://doi.org/10.1007/s10951-011-0231-3>.

- Kuhn, H.W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2), 83–97. <https://doi.org/10.1002/nav.3800020109>.
- Kühn, R., C. Weiß, H. Ackermann, S. Heydrich. (2024). Scheduling a single machine with multiple due dates per job. *Journal of Scheduling*, 27, 565–585. <https://doi.org/10.1007/s10951-024-00825-w>.
- Lawler, E.L. (1977). A “pseudopolynomial” algorithm for sequencing jobs to minimize total tardiness. *Annals of Discrete Mathematics*, 1, 331–342. [https://doi.org/10.1016/S0167-5060\(08\)70742-8](https://doi.org/10.1016/S0167-5060(08)70742-8).
- Lawler, E.L. (1982). A fully polynomial approximation scheme for the total tardiness problem. *Operations Research Letters*, 1(6), 207–208. [https://doi.org/10.1016/0167-6377\(82\)90022-0](https://doi.org/10.1016/0167-6377(82)90022-0).
- Lenstra, J.K., A.H.G. Rinnooy Kan, P. Brucker. (1977). Complexity of machine scheduling problems. *Annals of Discrete Mathematics*, 1, 343–362. [https://doi.org/10.1016/S0167-5060\(08\)70743-X](https://doi.org/10.1016/S0167-5060(08)70743-X).
- Mastrolilli, M. (2003). Efficient approximation schemes for scheduling problems with release dates and delivery times. *Journal of Scheduling*, 6, 521–531. <https://doi.org/10.1023/A:1026272526225>.
- Potts, C.N. (1980). Analysis of a heuristic for one machine sequencing with release dates and delivery times. *Operations Research*, 28(6), 1436–1441. <https://doi.org/10.1287/opre.28.6.1436>.
- Sgall, J. (2012). Open problems in throughput scheduling. In *Algorithms–ESA 2012, Lecture Notes in Computer Science* (Vol. 7501, pp. 2–11). Springer. https://doi.org/10.1007/978-3-642-33090-2_2.
- Smith, W.E. (1956). Various optimizers for single-stage production. *Naval Research Logistics Quarterly*, 3(1–2), 59–66. <https://doi.org/10.1002/nav.3800030106>.
- van den Akker, J.M., G. Diepen, J.A. Hoogeveen. (2010). Minimizing total weighted tardiness on a single machine with release dates and equal-length jobs. *Journal of Scheduling*, 13(6), 561–576. <https://doi.org/10.1007/s10951-010-0181-1>.
- Zhang, J., R. Chen, J. Yuan, C.T. Ng, T.C.E. Cheng. (2025). Bicriterion parallel-machine scheduling of equal-length jobs to minimize total tardiness and number of tardy jobs. *Journal of Combinatorial Optimization*, 50, Article 24. <https://doi.org/10.1007/s10878-025-01353-9>.